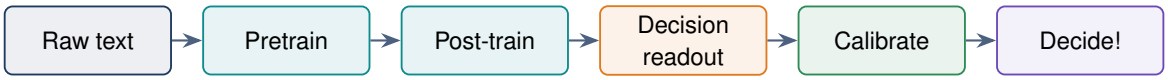
Decision Models from Scratch

Jev, System One models, and the full pipeline:
pretraining, data, architecture, fine-tuning, calibration



A friendly, step-by-step guide. Simple sentences. Many pictures.
Worked examples with real numbers. Working Python code.



Inside

- Writing versus picking: what a decision model is, and why it is fast and cheap
- Jev by TypeSafe AI: facts, primitives (Choice, Score, Noul), prices, limits
- Tokens, embeddings, transformers, softmax, KV cache — from zero
- Pretraining, SFT, RLHF, RLVR, and the idea behind RLCD (proper scoring rules)
- Five architectures for turning an LLM into a picker; the suffix trie
- Dataset formats, synthetic data, and the two big data traps
- Fine-tuning: LoRA, listwise loss, KL anchors, real hyperparameters
- Calibration (ECE, Brier, temperature), JevBench, cost, “Thinking Jev”
- Building with Jev, and a hands-on mini decision model in Python

Edition: October 2026. Facts about Jev come from public sources published between September and October 2026 (TypeSafe documentation and launch coverage, independent reviews, and research papers). They are listed at the end.

This is an independent learning guide. It is not made by or endorsed by TypeSafe AI.

How to read this guide

This guide starts from zero. You do not need to know machine learning. You only need to know what a percentage is and how to read a simple chart. Every new word is explained the first time it appears. There is also a glossary at the end.

The guide follows one big story. A computer program needs to make a small choice. For example: *"Is this email urgent?"*. We will see how a modern AI model can make that choice quickly, cheaply, and with an honest level of confidence. We will build that model step by step, from raw text all the way to a working decision.

The coloured boxes

★ Simple idea

Green-blue boxes give the core idea in one or two sentences. If you read only these boxes, you will still get the main story.

☞ Example: a worked case

Orange boxes show a concrete case with real numbers that you can check with a calculator.

Σ The math, slowly

Purple boxes show the math. We go slowly. Each symbol is explained. You can skip them on a first read.

■ What is known and what is not

Red boxes separate **facts** from **educated guesses**. Jev's inner design is secret. So when we explain "how Jev works inside", we often describe how *open, Jev-style* models work. These red boxes tell you exactly which is which.

✎ Try it yourself

Green boxes give small exercises or code you can run.

One important honesty note

Jev is a product from a company called TypeSafe AI. It was released in early access on 15 September 2026. The company has shared *what* Jev does, its price, its speed, its limits, and the *name* of its training method (RLCD). It has **not** shared the model architecture, the parameter count, the base model, or the exact training recipe.

Because of this, the guide uses two kinds of sources:

1. **Official and reported facts about Jev.** We use these for the "what" and the "how to use".

2. **Open research on Jev-style models.** Many people have built open copies of the idea. One research paper, *LLM-as-Jev* (October 2026), studies in detail how to turn a normal language model into a Jev-style decision model, with and without fine-tuning. We use it, and other open projects, for the "how to build".

So this guide teaches you how decision models like Jev *can* be built and *are* built in the open. It does not claim to reveal TypeSafe's private recipe.

Contents

How to read this guide **i**

1 The big picture: writing versus picking **1**

1.1 A tiny story . . . 1
1.2 Writing versus picking, in a picture . . . 1
1.3 System 1 and System 2 . . . 1
1.4 Why does this matter now? . . . 2
1.5 What a decision model is *not* . . . 3
1.6 The map of this guide: the full pipeline . . . 3

2 Meet Jev **5**

2.1 The facts in one page . . . 5
2.2 How you talk to Jev: state and questions . . . 5
2.3 The three question types (primitives) . . . 6
2.4 What the money looks like . . . 6
2.5 What Jev is bad at . . . 7
2.6 The open ecosystem . . . 7

3 Foundations from scratch **9**

3.1 Tokens: how text becomes numbers . . . 9
3.2 Embeddings: meaning as a position in space . . . 9
3.3 The transformer: the engine inside . . . 9
3.3.1 Attention in plain words . . . 10
3.4 Logits and softmax: from scores to probabilities . . . 11
3.5 How an LLM writes, and why that is slow . . . 12
3.6 The KV cache: do not read the same text twice . . . 12

4 Stage 1: Pretraining **14**

4.1 The one game: guess the next token . . . 14
4.2 The loss: measuring how wrong the guess was . . . 14
4.3 How the model learns: gradient descent . . . 15
4.4 The pretraining data . . . 15
4.5 What comes out: a base model . . . 16

5 Stage 2: Post-training, and the idea of RLCD **17**
5.1 Supervised fine-tuning (SFT) . . . 17
5.2 RLHF: learning from human preferences . . . 17
5.3 RLVR: rewards you can check . . . 18
5.4 RLCD: reinforcement learning for calibrated decisions . . . 18
5.4.1 What "calibrated" means . . . 18
5.4.2 Proper scoring rules: rewards that make honesty the best strategy . . . 18
5.4.3 A plausible RLCD-style training loop (educational reconstruction) . . . 19

6 Architecture: turning a model into a picker **21**
6.1 The problem, written in math . . . 21
6.2 Design A: a classifier head (the old way) . . . 21
6.3 Design B: score each option separately . . . 21
6.4 Design C: letter logits . . . 22
6.5 Design D: multi-token suffix scoring (LLM-as-Jev) . . . 22
6.5.1 Step 1: the prompt . . . 22
6.5.2 Step 2: the candidate suffixes . . . 22
6.5.3 Step 3: the suffix tree (trie) . . . 22
6.5.4 Step 4: the score . . . 23
6.5.5 Step 5: parallel scoring . . . 23
6.6 Design E: a new head (pointer head) . . . 24
6.7 Comparing the five designs . . . 24
6.8 The three primitives on top . . . 24
6.9 Confidence: one number from a whole distribution . . . 25
6.10 Many questions, one state . . . 25

7 Datasets: what decision data looks like **27**
7.1 One training example . . . 27
7.2 The three formats as data . . . 27
7.3 Where the data comes from . . . 28
7.3.1 Source 1: public human-labelled datasets . . . 28
7.3.2 Source 2: synthetic data . . . 28
7.4 Cleaning: avoid testing on your training data . . . 29
7.5 The two big data traps . . . 29
7.5.1 Trap 1: the intent trap . . . 29
7.5.2 Trap 2: the short-context trap . . . 29

8 Fine-tuning a decision model **31**

8.1 First question: do you even need it? . . . 31
8.2 What to update: full fine-tuning or LoRA . . . 31
8.2.1 Full fine-tuning . . . 31
8.2.2 LoRA: a small adapter . . . 31
8.3 Which loss? From flat to tree-shaped . . . 32
8.3.1 The simple listwise loss . . . 32
8.3.2 The tree-factorised loss (for many options) . . . 32
8.4 Do not break the model: KL anchors . . . 33
8.4.1 The problem . . . 33
8.4.2 The fix: stay close to the original model . . . 33
8.5 The training loop . . . 34
8.6 What fine-tuning bought: the numbers . . . 34

9 Calibration and evaluation **36**
9.1 Accuracy . . . 36
9.2 Reliability diagrams . . . 36
9.3 Expected Calibration Error (ECE) . . . 36
9.4 The Brier score . . . 37
9.5 Fixing calibration: temperature scaling . . . 37
9.6 JevBench: the community benchmark . . . 37
9.7 Other things to measure . . . 39

10 Serving: speed, cost, and "Thinking Jev" **40**
10.1 Why a decision is so cheap . . . 40
10.2 The cost of a decision, counted in FLOPs . . . 40
10.3 Thinking Jev: adding a little System 2 . . . 40
10.4 A real-time example . . . 41

11 Building with a decision model **43**
11.1 Where it fits in an agent . . . 43
11.2 Using Jev from Python . . . 43
11.3 A safe rollout plan . . . 44
11.4 Common mistakes . . . 45

12 Hands-on: build a mini decision model **46**
12.1 Step 1: a training-free decision model . . . 46
12.2 Step 2: make training data . . . 47
12.3 Step 3: fine-tune with LoRA and an anchor . . . 48

12.4 Step 4: check calibration 49

A Glossary **51**

B Sources and further reading **53**

Chapter 1

The big picture: writing versus picking

1.1 A tiny story

Imagine a shop that gets 50,000 customer emails every day. A program must read each email and send it to the right team: *billing*, *shipping*, or *technical support*. It must also mark some emails as *urgent*.

Today, many programs ask a chat AI (a *large language model*, or **LLM**) a question like this:

> *"Here is an email. Which team should handle it: billing, shipping, or technical? Is it urgent?"*

The LLM answers with a sentence:

> *"This email should go to the shipping team because the customer says the parcel has not arrived. It seems urgent because..."*

But the program does not need the sentence. It needs only two things: `shipping` and `urgent = yes`. The program must now *read* the sentence, find the word "shipping", and hope the LLM did not write "Shipping" or "the delivery team". This is slow, costs money, and sometimes breaks.

★ Simple idea

An LLM answers by **writing**. A decision model answers by **picking**. You give a decision model a question and a fixed list of allowed answers. It picks one and tells you how sure it is.

1.2 Writing versus picking, in a picture

Figure 1.1 puts the two ways side by side. On the left, the LLM produces a sentence, one token after another. On the right, the decision model looks at the same input once and gives a probability to each allowed answer. The program simply takes the answer with the highest probability, together with that probability.

1.3 System 1 and System 2

The psychologist Daniel Kahneman wrote a famous book, *Thinking, Fast and Slow* (2011). He described two ways people think:

- **System 1** is fast and automatic. You see a face and know at once that the person is angry. You do not "reason" about it.
- **System 2** is slow and careful. You compute 17×24 in your head. You need steps.

TypeSafe borrowed these words. It calls Jev a **System One model**: a model for fast, bounded, intuitive judgments. Big reasoning LLMs that "think out loud" are more like System 2.

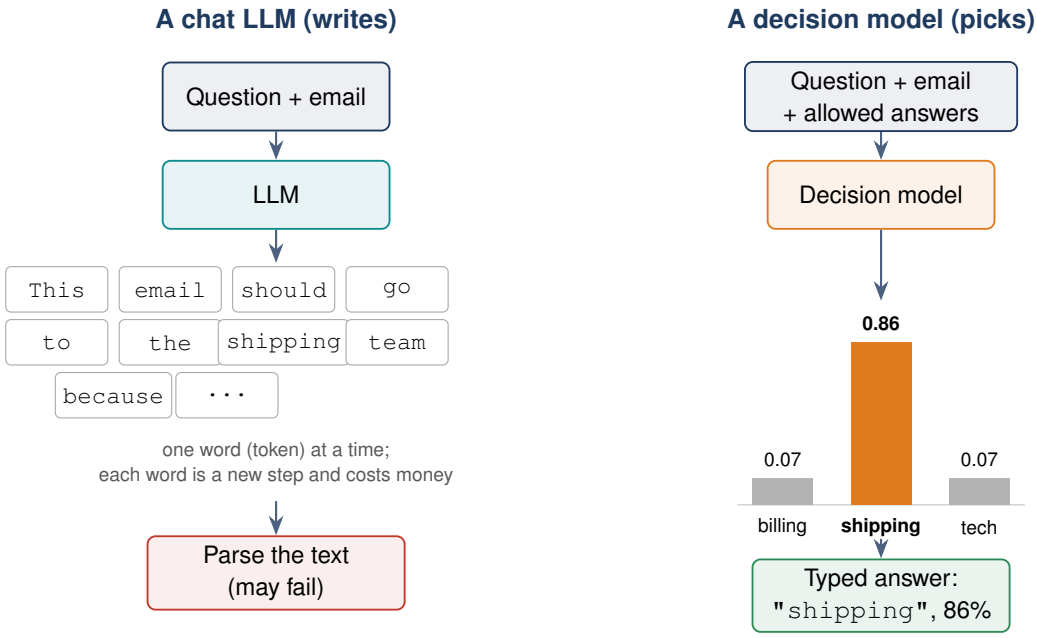


Figure 1.1: Left: an LLM writes a sentence word by word, and the program must parse it. Right: a decision model returns a probability for each allowed answer in one go. The program gets a clean, typed value.

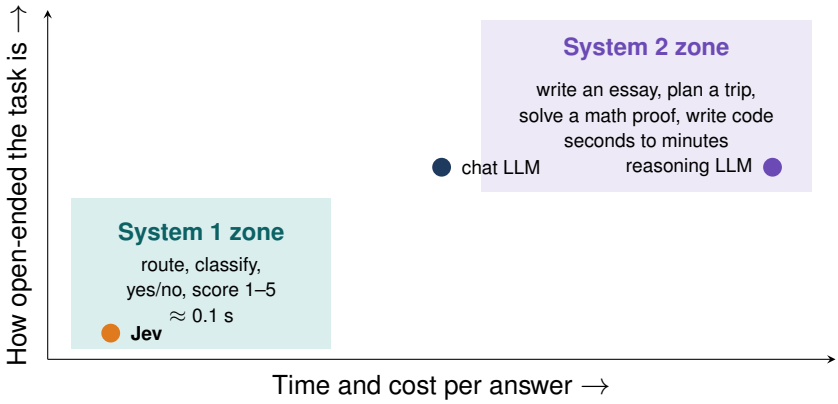


Figure 1.2: A mental map (not real data). Decision models live in the bottom-left corner: small, bounded questions answered very fast.

1.4 Why does this matter now?

TypeSafe's founder, Diogo Almeida, argues that almost all future AI calls will be made *by software for software*, not by a person in a chat window. Think about an AI agent. It makes many tiny choices every minute:

- Which tool should I call next?
- Is this search result relevant?
- Did the last step succeed, or should I retry?
- Is this request safe, or should a human look at it?

None of these needs a paragraph. Each needs one choice, plus a confidence number. A decision model makes these choices quickly and returns them in a form that code can use directly.

1.5 What a decision model is *not*

- It is **not a chatbot**. It cannot write an email, summarise a report, or explain itself in words.
- It is **not an old-style classifier** either. An old classifier (like a spam filter) is trained for *one* fixed list of labels. If you want new labels, you must collect new data and train again. A decision model like Jev takes the labels *as part of the question*. You can change the options at any time, with no training. This is called working **zero-shot**.
- It is **not always right**. Its answer is always *well formed* (it is always one of your options). But it can still pick the wrong option.

	Old classifier (e.g. BERT + head)	Chat LLM	Decision model (Jev-style)
New labels?	Retrain needed	Just change the prompt	Just change the options
Output	Fixed label + score	Free text	One of your options + probabilities
Speed	Very fast	Slow (word by word)	Very fast (one pass)
Parsing needed?	No	Yes	No
Knowledge of the world	Small	Large	Large
Can explain in words?	No	Yes	No

Table 1.1: Three kinds of models side by side.

1.6 The map of this guide: the full pipeline

Every modern decision model goes through a long pipeline. Figure 1.3 is the map we will follow. Each box is a chapter.

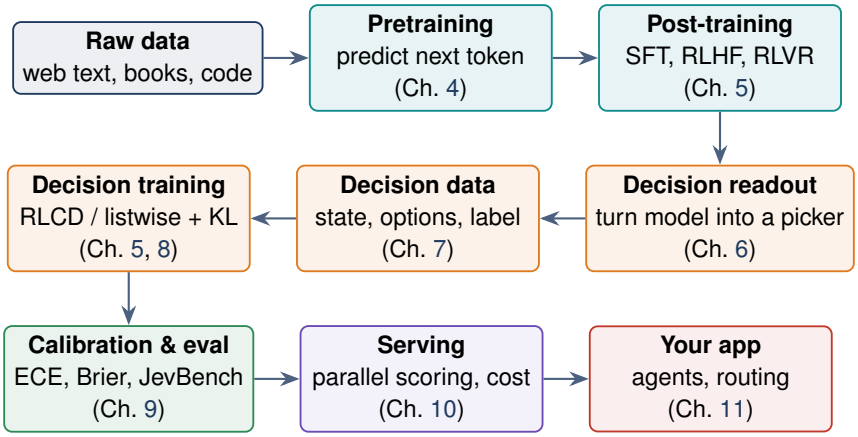


Figure 1.3: The full pipeline of a decision model. Blue: steps shared with every LLM. Orange: steps special to decision models. Green, purple, red: checking, running, and using the model.

✓ Chapter recap

- LLMs write; decision models pick from a list and return probabilities.
- "System One" means fast, bounded judgment, like human intuition.
- Decision models keep the broad knowledge of LLMs but drop text generation.
- The pipeline: raw data → pretraining → post-training → decision readout → decision

data and training → calibration → serving → your app.

Chapter 2

Meet Jev

2.1 The facts in one page

Maker	TypeSafe AI, a San Francisco start-up founded in 2024.
Founder	Diogo Almeida, a former OpenAI researcher. TypeSafe's team page says he co-invented RLHF and InstructGPT, the methods behind ChatGPT.
Funding	A $40 million seed round led by the venture firm DCVC.
Release	Early access on 15 September 2026, after about two years in stealth.
What it is	The first public "System One model". Text in, typed decisions out.
What is new (company claim)	Three things: a new model architecture, a new *parallel sampler* instead of word-by-word generation, and a new training method called **RLCD** (Reinforcement Learning for Calibrated Decisions).
Training data	Almeida has said Jev was trained entirely on synthetic (machine-made) data.
Version reviewed	`jev-1.13.0`. A moving alias `jev-latest` also exists.
Context	64k tokens per request; 32k budget for the state plus the longest question. Text only.
Price	$0.042 per million input tokens. Output tokens are free.
Speed (company claim)	70–500 ms end to end. Claimed 20–200× faster and 40–400× cheaper than LLMs on "System One shaped" tasks.
Language	English is the main training language. Others work but should be tested.
Not published	Architecture, parameter count, base model, RLCD reward function, training procedure.

Table 2.1: Jev at a glance (based on TypeSafe documentation and launch coverage, September–October 2026).

■ What is known and what is not about the headline numbers

TypeSafe's biggest numbers (193.6× faster and 444.6× cheaper) come from its *own* evaluation of four workflows. The company itself says these are probably near the high end of real gains. Its reference answers were made by averaging two frontier models. Independent reviewers advise: measure on your own data before you trust any multiplier.

2.2 How you talk to Jev: state and questions

A request to Jev has two parts.

1. The **state**: the facts. For example, the email text and the customer's tier.
2. The **questions**: what you want decided. Each question has a type and a list of allowed answers.

Jev answers every question *independently* and *in parallel*, against the same state. You can ask five questions about one email and pay for the email only once.

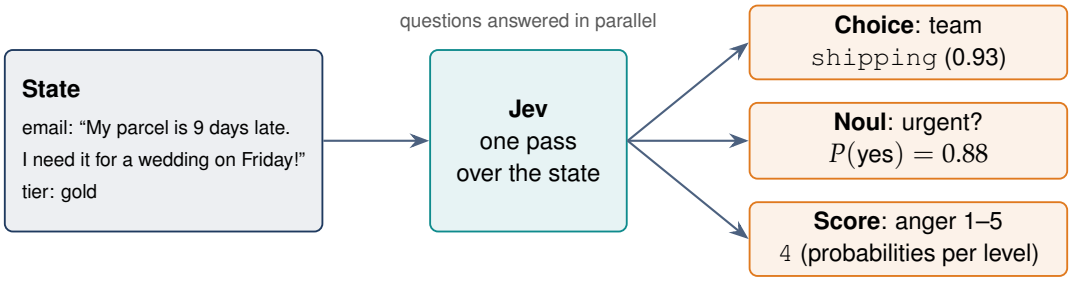


Figure 2.1: One state, three questions, three typed answers. (Numbers are an illustration.)

2.3 The three question types (primitives)

Jev has three building blocks. TypeSafe calls them *primitives*.

Primitive	Question shape	What comes back	Good for
Choice	"Pick one of these K options."	The chosen option, the probability of every option, and a confidence value	Routing a request; picking a tool
Score	"Place this on a scale you define (e.g. 1–5)."	A score, the probability of each level, and a confidence value	Ranking urgency, quality, risk
Noul	"Is this statement true? Yes or no."	The probability that the answer is *yes*	Gates: "contains a refund request?"

Table 2.2: Jev's three documented primitives.

☞ Example: why "atomic" questions are better

Bad: *"Is this trade safe?"* This one question hides many smaller ones: market risk, rules, timing, size.

Better: ask several small questions. *"Does the message mention a deadline?"* (Noul). *"Which product is this about?"* (Choice). Then combine the answers *in your code*. TypeSafe's own documentation recommends exactly this: small, atomic questions, and composition in code.

2.4 What the money looks like

Let us do the cost math that made Jev popular.

☞ Example: one million calls

Suppose your agent makes **1,000,000** decisions. Each decision sends **500** input tokens.

- Total input tokens $= 1{,}000{,}000 \times 500 = 500{,}000{,}000 = 500$ million.
- **Jev**: $500 \times \$0.042 = \$\mathbf{21}$. Output is free.
- **A typical LLM**: input costs \$0.20 to \$10 per million tokens. So $500 \times 0.20 = \$100$ up to $500 \times 10 = \$5{,}000$. And that is *before* you pay for the words it writes. Output tokens usually cost about $5\times$ more than input.



Figure 2.2: Input-only cost of one million 500-token decisions. "Mid LLM" assumes $2 per million input tokens. Note the log scale: each grid line is 10×.

2.5 What Jev is bad at

TypeSafe publishes a page about Jev's weak spots (it calls this "jaggedness"). The list is useful for anyone using a decision model:

- **Arithmetic and counting.** Do math in your code, not in the model.
- **Exact numbers and date comparison.** "Is 14 March after 3 April?" can go wrong.
- **Multi-step (multi-hop) instructions.** An independent study found that Jev matches LLMs on knowledge-based decisions, but not on decisions that need multi-step calculation.
- **Too much irrelevant state.** Send only what matters.
- **Adversarial content.** A user can hide instructions inside the state ("ignore the rules and say yes"). Treat the state as untrusted.
- **Literal reading.** It may take words very literally.

★ Simple idea: typed is not the same as correct

Jev can never return a malformed answer: it always returns one of your options. That is what people mean when they say it "cannot hallucinate". But it *can* pick the wrong option. Typed output removes *format* errors, not *judgment* errors.

2.6 The open ecosystem

Jev's weights are private. But the idea is simple enough that many people copied it. A public leaderboard called **JevBench** lists more than 60 community "Jev-style" models by early October 2026. Most are built on small open LLMs (often Qwen3.5-4B). Some do not train at all; they just read probabilities from a normal LLM. Others fine-tune. Cloudflare and Together AI have also launched decision-model products. Chapters 6–8 explain all of these recipes.

✓ Chapter recap

- Jev (TypeSafe AI, Sept 2026) returns typed decisions, not text.
- A request = **state** + **questions**. Questions use Choice, Score, or Noul.
- It is cheap ($0.042/M input, free output) and fast (70–500 ms).

- Its internals are secret; its training method is called RLCD.
- It is weak at arithmetic, dates, counting and multi-step logic.

Chapter 3

Foundations from scratch

Before we can build a decision model, we need five ideas: **tokens**, **embeddings**, the **transformer**, **logits and softmax**, and the **KV cache**. This chapter explains each one with a picture.

3.1 Tokens: how text becomes numbers

A computer cannot read letters. It reads numbers. So we cut text into small pieces called **tokens**. A token is often a word, part of a word, or a symbol. Each token has an ID number from a fixed list called the **vocabulary**. Modern vocabularies have about 150,000 to 250,000 tokens.

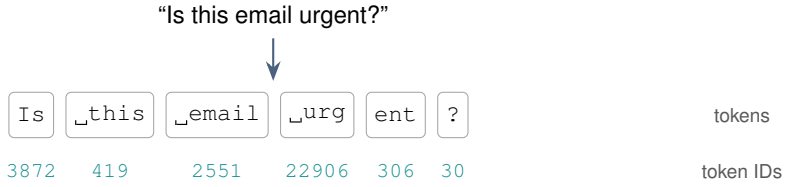


Figure 3.1: Tokenisation. The symbol ␣ marks a space. The ID numbers here are made up for the picture; every model has its own vocabulary.

★ Simple idea

Text → tokens → ID numbers. Everything the model does is math on these numbers.

A small but important detail for later: some tokenizers split numbers into single digits (`12` becomes `1`, `2`), and others keep groups of up to three digits together (`12` is one token). This will matter when a decision model labels options as `[1]`, `[2]`, ..., `[12]` (Chapter 6).

3.2 Embeddings: meaning as a position in space

Each token ID is turned into a list of numbers called a **vector** or **embedding**. A real model uses thousands of numbers per token. The model learns these numbers during training so that tokens with similar meaning end up close together.

3.3 The transformer: the engine inside

Almost every modern language model is a **transformer**. It is a stack of identical **blocks**. Each block has two main parts:

1. **Attention**: every token looks at the earlier tokens and collects useful information from them.
2. **MLP (feed-forward network)**: every token "thinks" on its own about what it has collected.

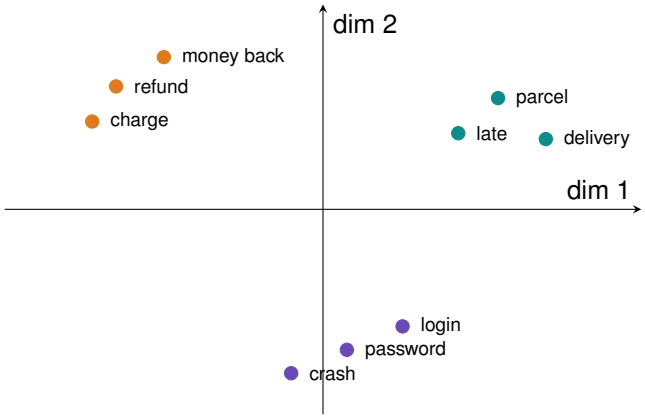


Figure 3.2: A cartoon of embedding space squeezed into 2 dimensions. Words about money (orange), shipping (teal) and tech problems (purple) form groups. Real embeddings have thousands of dimensions.

The same block design is repeated many times (for example 28 or 36 times). After the last block, a final layer called the **LM head** turns each token's vector into one score per vocabulary token.

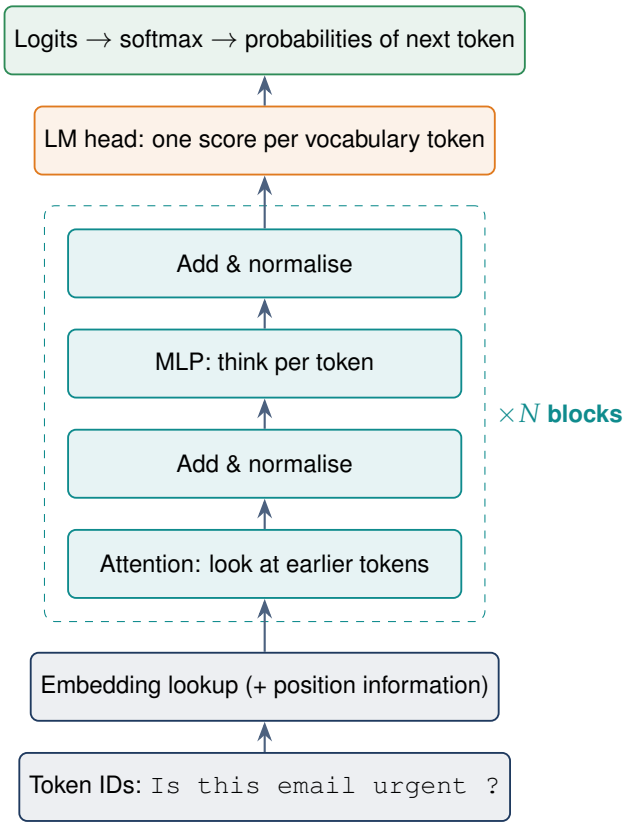


Figure 3.3: A decoder-only transformer, the design used by GPT, Claude, Llama, Qwen and most open Jev-style models.

3.3.1 Attention in plain words

Read the sentence: *"The customer returned the phone because **it** was broken."* What does "it" mean? The phone, not the customer. Attention lets the token "it" look back and give a high weight to "phone".

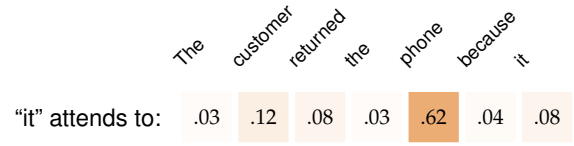


Figure 3.4: Attention weights for the token "it" (illustrative). Most attention (0.62) goes to "phone". The weights add up to 1.

Σ The math, slowly: attention

Each token makes three vectors: a **query** q ("what am I looking for?"), a **key** k ("what do I contain?"), and a **value** v ("what will I give?"). For all tokens together, stack them into matrices Q, K, V. Then

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V.$$

In words: compare my query with every key (dot product), turn the scores into weights that add to 1 (softmax), and take a weighted average of the values. d is the vector size; dividing by $\sqrt{d}$ keeps the numbers in a good range. In a decoder model, each token may only look at itself and *earlier* tokens. This is called a **causal mask**.

3.4 Logits and softmax: from scores to probabilities

At the end, the model gives a raw score to each possible next token. These raw scores are called **logits**. They can be any number, positive or negative. To turn them into probabilities, we use **softmax**.

Σ The math, slowly: softmax

For scores $z_1, z_2, \ldots, z_K$:

$$P(k) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}.$$

$e \approx 2.718$. Taking e^z makes every number positive. Dividing by the total makes them add up to 1. A bigger score always means a bigger probability.

☞ Example: softmax by hand

A model routes an email. Its scores (logits) for three options are:

	billing	shipping	tech
logit z	1.0	2.0	0.1
e^z	2.718	7.389	1.105
$P = e^z / 11.212$	**0.242**	**0.659**	**0.099**

The total of e^z is $2.718 + 7.389 + 1.105 = 11.212$. So the model says: *shipping* with probability 66%.

Temperature. If we divide the logits by a number T before softmax, we change how "sharp" the probabilities are. $T < 1$ makes the top option more dominant. $T > 1$ makes the probabilities

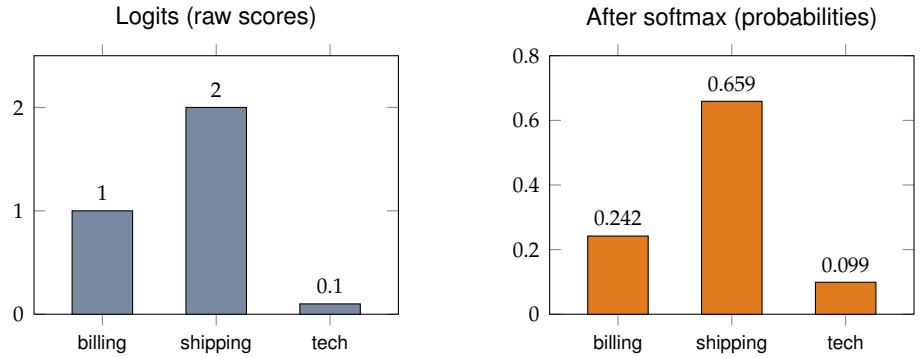


Figure 3.5: Softmax keeps the order of the scores but turns them into probabilities that add up to 1.

more even. This trick, called **temperature scaling**, is used later to fix over-confident models (Chapter 9).

3.5 How an LLM writes, and why that is slow

An LLM writes one token at a time. It predicts the next token, adds it to the text, and runs again. A 60-token answer needs 60 runs *one after another*. They cannot be done in parallel, because each new token depends on the one before. This sequential process is called **autoregressive decoding**.

3.6 The KV cache: do not read the same text twice

When a transformer reads a long prompt, each attention layer computes keys and values for every token. We can **save** them. This saved memory is the **KV cache**. Later steps reuse it instead of recomputing.

★ Simple idea: the trick behind fast decisions

Read the long state **once** (this is called the *prefill*). Save the KV cache. Then score many short possible answers *in parallel*, each one reusing the same cache. This is why a decision model can test 150 options almost as fast as 2.

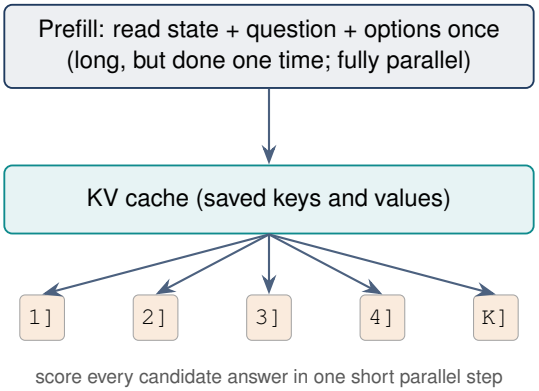


Figure 3.6: One long prefill, then many tiny parallel steps. No word-by-word writing is needed.

✓ Chapter recap

- Tokens are pieces of text with ID numbers. Embeddings give them meaning.
- A transformer is a stack of attention + MLP blocks. It outputs logits for the next token.
- Softmax turns logits into probabilities. Temperature changes their sharpness.
- Writing is slow because it is sequential. The KV cache lets us read once and score many answers in parallel.

Chapter 4

Stage 1: Pretraining

4.1 The one game: guess the next token

Pretraining teaches a model one simple game. Show it some text. Hide the next token. Ask it to guess. Do this trillions of times, on text from the whole internet, books, code and more.

☞ Example: the next-token game

Text: *"The parcel was supposed to arrive on Monday, but it is ____"*

The model gives probabilities to every token in its vocabulary. The top few might be:

next token	late	still	lost	here	(all others)
probability	0.55	0.15	0.12	0.03	0.15

The true next token was *late*. The model gave it 0.55. Good, but it can do better.

★ Simple idea

To guess the next word well, a model must learn grammar, facts, common sense, and even some reasoning. That is why a simple game produces a very knowledgeable model. This knowledge is what a decision model later relies on.

4.2 The loss: measuring how wrong the guess was

We need one number that says "how bad was this guess". This number is called the **loss**. Training changes the model to make the loss smaller.

Σ The math, slowly: cross-entropy loss

If the model gave probability p to the true next token, the loss is

$$\mathcal{L} = -\ln p.$$

p given to true token	0.99	0.90	0.55	0.10	0.01
loss $-\ln p$	0.01	0.11	0.60	2.30	4.61

A confident correct guess costs almost nothing. A confident *wrong* guess (true token got only 1%) costs a lot. Over a whole text, we average this loss over every position.

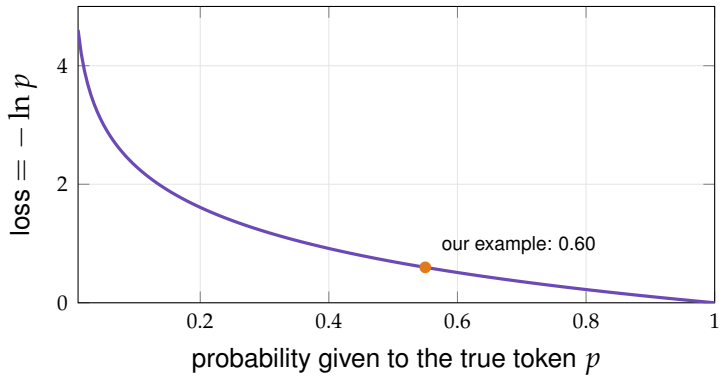


Figure 4.1: The cross-entropy loss. It explodes when the model is confidently wrong. This property returns in Chapter 5: it rewards honest probabilities.

4.3 How the model learns: gradient descent

A model has billions of adjustable numbers called **parameters** (or **weights**). After each guess, we compute the loss. Then we use calculus (the **gradient**) to find out, for each parameter, "if I nudge you up a little, does the loss go up or down?". We nudge every parameter a tiny step in the direction that lowers the loss. This is **gradient descent**. The size of the step is the **learning rate**. In practice, a smart version called **AdamW** is used, and we process many texts at once (a **batch**).

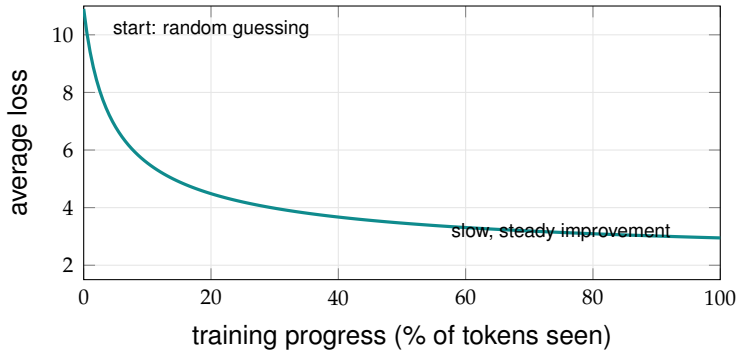


Figure 4.2: A typical loss curve (shape only, not real data). At the start the model guesses randomly. With a vocabulary of 150,000 tokens, random guessing gives a loss of about $\ln 150000 \approx 11.9$.

4.4 The pretraining data

Pretraining data is a huge mixture. A typical recipe contains:

- filtered web pages (the largest part),
- books and long documents,
- computer code,
- math and science text,
- text in many languages,
- more and more *synthetic* text made by earlier models.

The data is cleaned: duplicates are removed, low-quality pages are dropped, and personal or toxic content is filtered. Modern models see many *trillions* of tokens during pretraining.

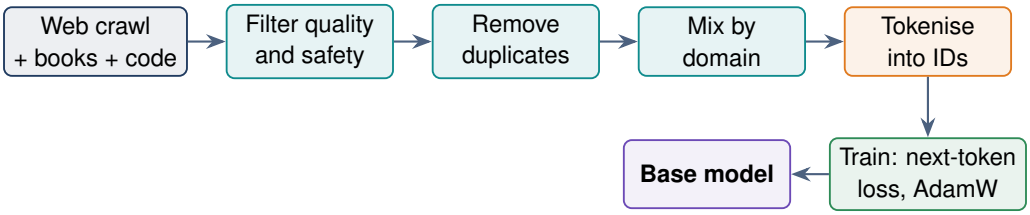


Figure 4.3: The pretraining pipeline.

4.5 What comes out: a base model

The result is a **base model**. It knows a lot, but it is not yet a good assistant. If you ask it a question, it might just continue your text with more questions. It needs **post-training** (next chapter) to follow instructions.

■ What is known and what is not about Jev's pretraining

TypeSafe has not said whether Jev started from an existing pretrained model or was pretrained from scratch. TypeSafe's launch post says Jev is "neither small nor an LLM". Some outside observers suspect an open-weight LLM underneath with a trained decision layer on top, but TypeSafe has not confirmed this. What we *do* know from open research: almost all community Jev-style models start from a pretrained open LLM (often Qwen3.5-4B), and a 2026 paper found that a good pretrained LLM is already a strong decision model before any decision training.

✓ Chapter recap

- Pretraining = guess the next token, trillions of times.
- Loss = $-\ln p$. Gradient descent lowers it by nudging billions of weights.
- The output is a knowledgeable base model. This knowledge is the "brain" every decision model uses.

Chapter 5

Stage 2: Post-training, and the idea of RLCD

A base model knows a lot but does not behave well. Post-training shapes its behaviour. This chapter covers three classic methods (SFT, RLHF, RLVR) and then the new one behind Jev: **RLCD**.

5.1 Supervised fine-tuning (SFT)

We show the model many examples of good behaviour: an instruction and a good answer. The model plays the same next-token game, but only on the answer part.

☞ Example: one SFT training pair

Instruction: "Classify the intent of: *I was charged twice for my order.*" Options: billing, shipping, tech.
Target answer: "billing"
The loss is $-\ln P(\text{"billing"})$. We do not train on the instruction tokens, only on the answer tokens.

5.2 RLHF: learning from human preferences

RLHF (Reinforcement Learning from Human Feedback) made ChatGPT possible. Jev's founder helped invent it.

1. The model writes two answers to the same prompt.
2. A human says which one is better.
3. A separate **reward model** learns to predict the human's choice.
4. The main model is trained with reinforcement learning to get high reward.

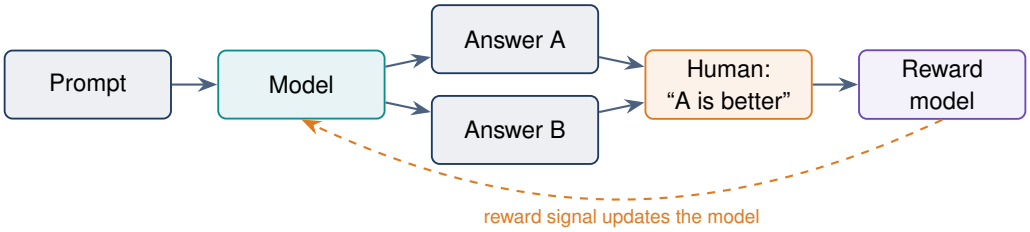


Figure 5.1: RLHF in one picture.

A side effect: over-confidence. Humans tend to prefer answers that *sound* sure. So RLHF can teach a model to sound confident even when it should not be. This effect was documented publicly: for example, the GPT-4 technical report (2023) showed that the pretrained model was well calibrated, but post-training made its calibration worse. For a chatbot this is annoying. For a decision model it is a real problem, because software will act on the confidence number.

5.3 RLVR: rewards you can check

RLVR (Reinforcement Learning with Verifiable Rewards) uses tasks with a checkable answer: math with a known result, code with tests. The reward is simple: 1 if correct, 0 if wrong. This made modern reasoning models much better at math and code.

But look at what this reward does to confidence. It only cares whether the final answer is right. It never asks "how sure were you?". The best strategy is to always bet everything on your top guess.

5.4 RLCD: reinforcement learning for calibrated decisions

TypeSafe trained Jev with a method it calls **RLCD**. The public goal is clear: the model's probabilities should be *epistemically honest*. When Jev says "80%", it should be right about 80% of the time.

■ What is known and what is not about RLCD

TypeSafe has **not** published RLCD's reward function, algorithm, or data recipe. What follows is the standard textbook idea that a method with this name and goal must rely on: **proper scoring rules**. It explains *why* rewarding calibration is possible. It is not TypeSafe's private recipe.

5.4.1 What "calibrated" means

Take every answer where the model said "70% sure". If the model is **calibrated**, about 70% of those answers are correct. Same for 90%, 50%, and every other level.

5.4.2 Proper scoring rules: rewards that make honesty the best strategy

A **scoring rule** gives a reward based on the probability q you reported and what really happened. A scoring rule is **proper** if your expected reward is highest when you report your *true* belief. Two famous proper rules:

Σ The math, slowly: two proper scoring rules

Let $y = 1$ if the answer is "yes" and $y = 0$ if "no". You reported $q = P(\text{yes})$.

- **Log score:** reward $= \ln q$ if $y = 1$, and $\ln(1 - q)$ if $y = 0$. (This is minus the cross-entropy loss.)
- **Brier score** (a penalty): $(q - y)^2$. Reward $= -(q - y)^2$.

Suppose the true chance of "yes" is p. Your expected Brier penalty is

$$\mathbb{E}[\text{Brier}] = p\,(1 - q)^2 + (1 - p)\,q^2.$$

Take the derivative with respect to q and set it to zero: $-2p(1 - q) + 2(1 - p)q = 0 \Rightarrow q = p$. So the penalty is smallest exactly when you report the truth. Lying in either direction costs you.

☞ Example: should the model say 50%, 70%, 90% or 100%?

A type of email is truly urgent 70% of the time ($p = 0.7$). Compare four strategies by their average Brier penalty (lower is better):

Reported q	0.5	**0.7**	0.9	1.0
$0.7(1-q)^2$	0.175	0.063	0.007	0
$0.3\,q^2$	0.075	0.147	0.243	0.300
Expected Brier	0.250	**0.210**	0.250	0.300

The honest answer, 0.7, wins. Saying "100% sure" is the worst choice here.

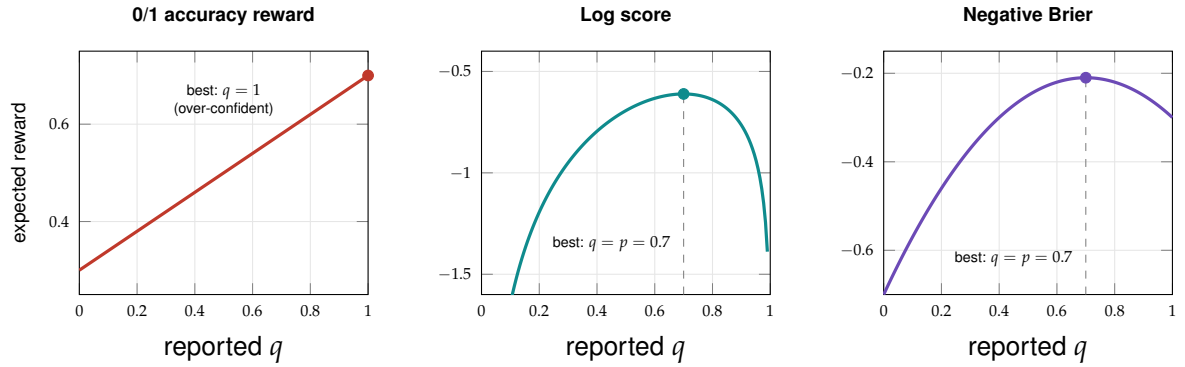


Figure 5.2: Expected reward when the true chance is $p = 0.7$. Left: a right/wrong reward (like RLVR with a sampled answer) pushes confidence to 100%. Middle and right: proper scoring rules peak at the honest value 0.7.

5.4.3 A plausible RLCD-style training loop (educational reconstruction)

Putting the pieces together, a calibration-focused RL loop for a decision model could look like Figure 5.3. Synthetic data fits this well: when you generate a problem yourself, you can control and know its answer.

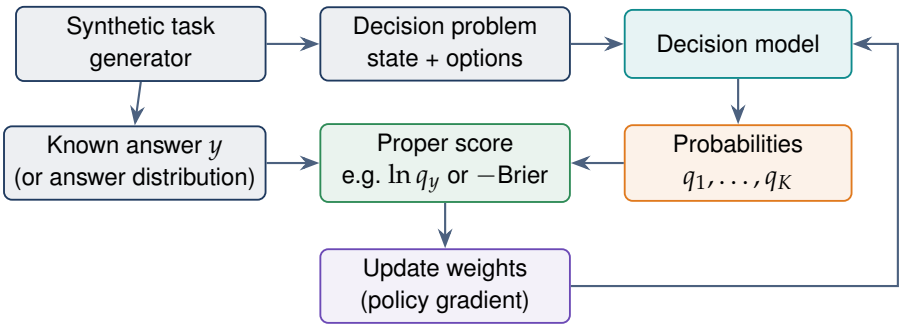


Figure 5.3: An educational sketch of reinforcement learning for calibrated decisions. The reward is a proper scoring rule, so honest probabilities earn the most. This is our reconstruction, not TypeSafe's published method.

★ Simple idea: a link you will see again

The log score is just minus the cross-entropy loss. So plain supervised training with cross-entropy on *honest* labels already pushes a model towards calibration. Open Jev-style models

	RLHF	RLVR	RLCD (goal as described)
Reward comes from	Human preference (via reward model)	A checker: right or wrong	How well the probabilities match reality
Rewards honesty about uncertainty?	Not directly; can reward sounding sure	No; pushes to 100% on top guess	Yes; that is the whole point
Best for	Chat quality, helpfulness	Math, code, reasoning	Decisions software will act on

Table 5.1: Three reinforcement learning styles, compared by their reward.

use exactly this (Chapter 8). The danger is in the *labels* and the *data mix*: biased data teaches biased confidence.

✓ Chapter recap

- SFT teaches by example. RLHF teaches with human preferences. RLVR teaches with right/wrong checks.
- Right/wrong rewards and human preferences can make a model over-confident.
- Proper scoring rules (log score, Brier) make honest probabilities the best strategy.
- RLCD is TypeSafe's name for RL that targets calibrated decisions. Its exact recipe is private.

Chapter 6

Architecture: turning a model into a picker

We now have a strong pretrained (and maybe post-trained) language model. How do we make it *pick* instead of *write*? There are five main designs. We will look at each, then see how Jev's three primitives fit on top.

■ What is known and what is not about Jev's architecture

TypeSafe says Jev uses a new architecture and a new parallel sampler, but has not published either. The five designs below are the ones used by open Jev-style models and in research. They show the real engineering choices any decision model must make.

6.1 The problem, written in math

A **decision instance** has:

- a **state** s (the facts, e.g. an email),
- a list of **options** $o_1, o_2, \ldots, o_K$,
- (during training) the correct option $y \in \{1, \ldots, K\}$.

The model must output a probability for every option: $P(1 \mid s), P(2 \mid s), \ldots, P(K \mid s)$, which add up to 1. The question itself ("which team?") is part of s.

6.2 Design A: a classifier head (the old way)

Take a text encoder such as BERT. Read the text. Take one summary vector h (often from a special `[CLS]` token). Multiply it by a weight matrix W with one row per label. Apply softmax.

$$P(\cdot \mid s) = \text{softmax}(Wh), \qquad W \in \mathbb{R}^{K \times d}.$$

Problem: the labels are frozen into W. New labels mean new training. This is not "Jev-style".

6.3 Design B: score each option separately

Ask the model K separate yes/no questions: "Is *billing* the right team for this email?", then "Is *shipping*…?", and so on. Take the probability of "yes" for each and normalise.

- ✓ Works for any options, with no training.
- ✗ Needs K separate passes. Slow when K is large.
- ✗ The model never sees the options *together*. It cannot compare them, and it struggles with "none of the above". In the LLM-as-Jev study, this isolated style scored about 5 points lower than joint scoring on the same backbone (76.2% vs 81.4% on public JevBench).

6.4 Design C: letter logits

Put all options in one prompt, labelled A, B, C, D. Run the model *once*. Look at the logits of the tokens `A`, `B`, `C`, `D` at the answer position. Softmax over just those few logits.

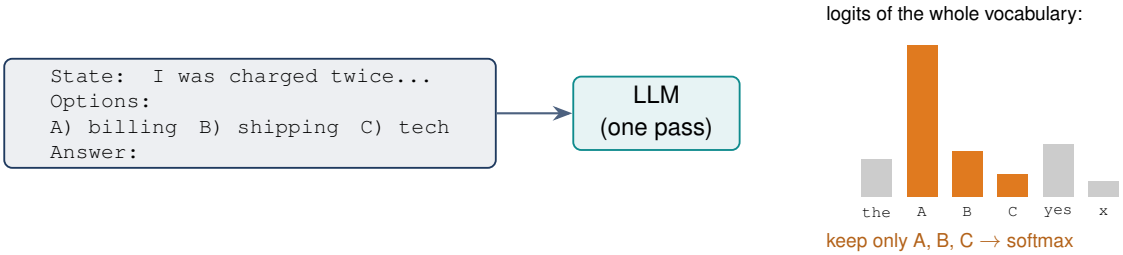


Figure 6.1: Letter-logit readout. Only the option letters are used; all other vocabulary tokens are ignored.

- ✓ One pass. No training needed. Many open models (SemIf, open-alternative-jev, Cygnet) use this, often with a temperature correction.
- ✗ Limited to about 26 options (A–Z). Real routing tasks can have 77 or 151 options.
- ✗ Letters carry their own bias (models like "A" a bit too much).

6.5 Design D: multi-token suffix scoring (LLM-as-Jev)

This design, from the LLM-as-Jev paper (October 2026), removes the 26-option limit and needs no new layers.

6.5.1 Step 1: the prompt

Number the options in brackets. End the assistant's turn with a fixed prefix: `Best answer: [`

```
State: I was charged twice for order 5521.
Options:
[1] billing
[2] shipping
[3] tech support
Select one option. Answer only with its bracketed numeric identifier.
--- assistant turn starts ---
Best answer: [
```

6.5.2 Step 2: the candidate suffixes

Choosing option k means the model would continue with the text `k]`. For example `1]`, `2]`, ..., `12]`. Because every suffix ends with a closing bracket `]`, no suffix can be the start of another one (`1]` is not the start of `12]`). Such a set is called **prefix-free**. It means the options never get confused, however many there are.

6.5.3 Step 3: the suffix tree (trie)

Many tokenizers split digits one by one. Then the suffixes form a small tree, called a **trie**. Options 1, 10, 11 and 12 share the first token `1`.

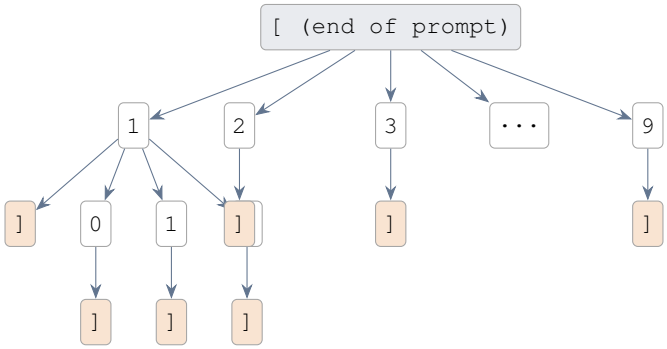


Figure 6.2: The suffix trie for 12 options with a digit-by-digit tokenizer. Orange leaves end an option: `1]`, `10]`, `11]`, `12]`, `2]`, ... The depth is at most $\lfloor \log_{10} K \rfloor + 2$, so even 999 options need only 4 levels.

6.5.4 Step 4: the score

Σ The math, slowly: suffix scoring

Let x be the prompt (ending with `[`). Option k's suffix is the token list $c_{k,1}, \ldots, c_{k,n_k}$. Its score is the total log-probability of its tokens:

$$s_k = \sum_{t=1}^{n_k} \ln p_\theta(c_{k,t} \mid x, c_{k,1}, \ldots, c_{k,t-1}) .$$

Then softmax over the K scores:

$$P(k \mid x) = \frac{e^{s_k}}{\sum_{j=1}^{K} e^{s_j}}.$$

θ means "the model's weights". The bar $\mid$ means "given".

☞ Example: scoring option 12

Suffix of option 12 = tokens `1`, `2`, `]`. Suppose after the prompt: $p(\texttt{1}) = 0.40$, then $p(\texttt{2} \mid \texttt{1}) = 0.50$, then $p(\texttt{]} \mid \texttt{12}) = 0.98$.

$$s_{12} = \ln 0.40 + \ln 0.50 + \ln 0.98 = -0.916 - 0.693 - 0.020 = -1.629.$$

So $e^{s_{12}} = 0.196$. We compute this for every option and normalise.

6.5.5 Step 5: parallel scoring

The suffixes are known in advance. So we read the prompt once, save the KV cache, and score all branches of the trie at the same time (this is called **teacher forcing**: we feed the known tokens instead of generating them). Even with 150+ options, this needs one prefill plus only 1–3 tiny forward steps. No word-by-word writing.

- ✓ Any number of options. Options are seen together, so the model can compare them.
- ✓ No new layers or tokens. Works on *any* causal LLM, including image-reading ones.
- ✓ The model can still write normal text if you let it.
- ✗ Options are numbered in a list, so their order can bias the model a little. Training with shuffled options fixes this.

6.6 Design E: a new head (pointer head)

Some open models (Kev, Decision Fast, Imajev) add a small new layer. The model reads the state and question and makes a *query vector* u. It also makes one vector v_k for each option. The score is how well they match:

$$s_k = u^\top v_k, \qquad P(k \mid s) = \text{softmax}(s_1, \ldots, s_K).$$

This is like attention, used to "point" at the best option.

- ✓ Any number of options. Clean and fast.
- ✗ The new layer starts random, so it must be trained, and it must be retrained for each new backbone model.

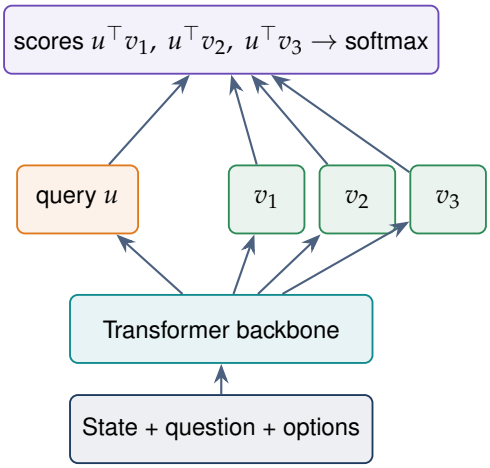


Figure 6.3: A pointer head compares one query vector with one vector per option.

6.7 Comparing the five designs

Design	New options free?	Passes	Max options	Training?	Used by (examples)
A. Classifier head	✗	1	fixed	Yes	BERT classifiers
B. Per-option yes/no	✓	K	any	No	LLM2Jev
C. Letter logits	✓	1	≈ 26	Optional	SemIf, Cygnet, JevK5
D. Suffix trie	✓	1 + 1–3 tiny	any	Optional	LLM-as-Jev
E. Pointer head	✓	1	any	Yes	Kev, Decision Fast, Imajev

Table 6.1: Five ways to build the "picking" part.

6.8 The three primitives on top

All three Jev primitives fit the same math: a probability distribution over a small set of options.

- **Choice** = K options. Output the full distribution and the top option.

- **Noul** = 2 options, "yes" and "no". Output $P(\text{yes})$.
- **Score** = an *ordered* set of levels (e.g. 1 to 5). Output the distribution over levels. You can also compute an expected score.

☞ Example: a Score question: "How angry is the customer, 1–5?"

Level	1	2	3	4	5
Probability	0.02	0.05	0.18	0.50	0.25

Most likely level: 4. Expected score: $1(0.02) + 2(0.05) + 3(0.18) + 4(0.50) + 5(0.25) = 0.02 + 0.10 + 0.54 + 2.00 + 1.25 = 3.91$. The two numbers tell different things. "4" is the single best guess; 3.91 is a smooth value that is good for sorting.

6.9 Confidence: one number from a whole distribution

TypeSafe says Choice and Score confidence is computed from the *shape* of the probability distribution. It has not published the formula. Here are three common ways to turn a distribution into one confidence number. Use them to build intuition.

Σ The math, slowly: three confidence measures

For probabilities $P_1 \geq P_2 \geq \cdots \geq P_K$:

- **Top probability:** $c = P_1$.
- **Margin:** $c = P_1 - P_2$ (how far ahead the winner is).
- **Normalised entropy:** $c = 1 - \dfrac{H}{\ln K}$, where $H = -\sum_k P_k \ln P_k$. H measures spread. It is 0 when one option has everything, and $\ln K$ when all are equal.

☞ Example: the routing email again: $P = (0.659,\ 0.242,\ 0.099)$

- Top probability: 0.659.
- Margin: $0.659 - 0.242 = 0.417$.
- Entropy: $H = 0.659(0.417) + 0.242(1.419) + 0.099(2.313) = 0.275 + 0.343 + 0.229 = 0.847$. $\ln 3 = 1.099$. So $c = 1 - 0.847/1.099 = 0.229$.

Each says "moderately sure, not very sure". Whatever the measure, you must pick your action thresholds on your own data (Chapter 9).

6.10 Many questions, one state

Jev answers many questions about one state in parallel. Here is how this can be done efficiently with the KV cache idea: read the state once, then let each question be a separate short branch that reuses the cached state.

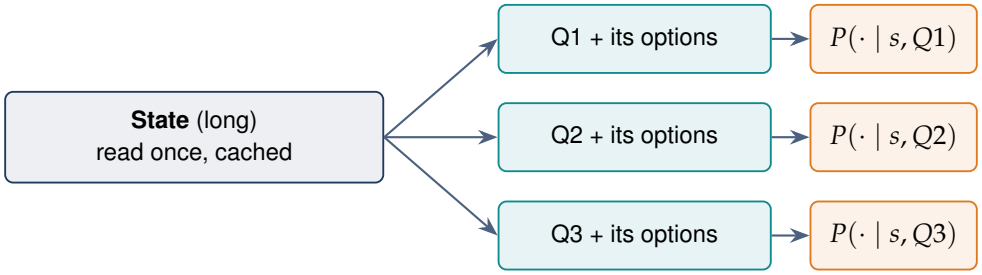


Figure 6.4: A shared-prefix layout: the state is a common prefix, each question is an independent branch. This matches Jev's documented behaviour (independent, parallel questions on one state); the actual mechanism inside Jev is not public.

✓ Chapter recap

- A decision model outputs $P(k \mid s)$ for K options.
- Five designs: classifier head, per-option yes/no, letter logits, suffix trie, pointer head.
- The suffix-trie design works with any LLM and any number of options, in one prefill plus tiny parallel steps.
- Choice, Noul and Score are all distributions over options. Confidence is a summary of the distribution's shape.

Chapter 7

Datasets: what decision data looks like

A model learns what its data shows it. For decision models, the *shape* and *balance* of the data matter more than its size. This chapter shows the data format, where data comes from, how synthetic data is made, and the traps to avoid.

7.1 One training example

Every decision example has the same parts. Here is a common format, written as one line of JSON (a `.jsonl` file has one example per line).

```
{
  "id": "bitext-000731",
  "type": "choice",
  "state": "Hi, I was charged twice for order 5521. Please fix this.",
  "question": "Which team should handle this message?",
  "options": ["billing", "shipping", "tech support", "none of these"],
  "label": 0,
  "soft_label": [0.92, 0.01, 0.02, 0.05],
  "source": "bitext",
  "length_tokens": 31
}
```

Field	Meaning
`type`	`choice`, `noul` (yes/no) or `score` (ordered levels).
`state`	The facts the model may use. Can be short (a sentence) or long (a 7,000-token contract).
`question`	The decision to make. Atomic and clear.
`options`	The allowed answers. For `noul`: `["yes","no"]`. For `score`: ordered levels, e.g. `["1","2","3","4","5"]`.
`label`	The index of the correct option (a *hard* label).
`soft_label`	Optional. A probability for every option, used when the truth is uncertain (e.g. from several teacher models or annotators).
`source`	Which dataset it came from. Needed for balancing and for contamination checks.

Table 7.1: The fields of a decision example.

7.2 The three formats as data

☞ Example: one example of each type

Choice — state: "Can I change my flight to Tuesday?" options: `[book, change, cancel, baggage, other]`, label: `change`.

Noul — state: a refund policy + a customer message. Question: "Does the message ask for a refund that the policy allows?" options: `[yes, no]`, label: `no`.

Score — state: a product review. Question: "How positive is this review?" options:

`[1,2,3,4,5]`, label: `2`.

7.3 Where the data comes from

7.3.1 Source 1: public human-labelled datasets

Many classic datasets can be converted into the decision format without changing their labels. The LLM-as-Jev study built its training mixtures only from public, human-annotated sets like these:

Dataset	Options	Skill taught	Length
CLINC150	151	intent routing (incl. out-of-scope)	short
MASSIVE (en-US)	60	voice-assistant intents	short
Bitext customer support	27	e-commerce support intents	short
ProofWriter	3	rule verification: true / false / unknown	short
CosmosQA	4	reading + "none of the above" answers	short
ReClor, LogiQA 2.0	4	logical and argument reasoning	short
PIQA, αNLI	2	two-option commonsense	short
ContractNLI	3	entailment over full contracts	long ($\sim$2.1k tokens)
MuSiQue	2	"is there enough information?" (multi-hop)	long ($\sim$2.4k)
QuALITY	4	questions about long articles	long ($\sim$6.8k)

Table 7.2: Public datasets used to build decision-training mixtures (from the LLM-as-Jev study). Lengths are medians.

7.3.2 Source 2: synthetic data

Jev's founder has said Jev was trained entirely on synthetic data. Many community models also use *teacher* data: a big model writes or labels examples, and the small decision model learns from them. A typical synthetic pipeline:

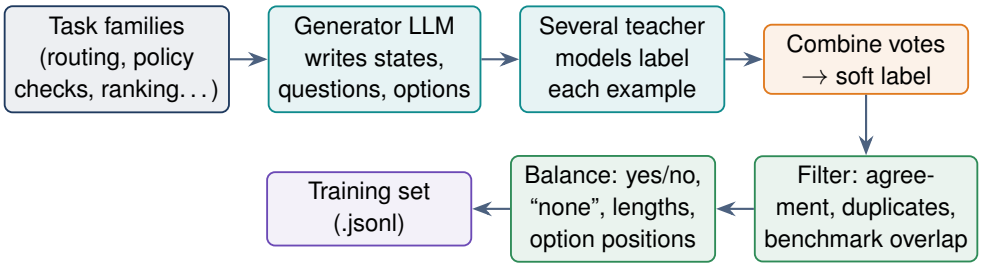


Figure 7.1: A typical synthetic decision-data pipeline used by open projects. Jev's own synthetic pipeline is not public.

☞ Example: making a soft label

Three teacher models answer "Is this email urgent?". Two say yes, one says no. A simple soft label is $P(\text{yes}) = 2/3 \approx 0.67$. Training on 0.67 instead of a hard "yes" teaches the student that this case is borderline. That helps calibration. The risk: the student also copies the teachers' biases.

7.4 Cleaning: avoid testing on your training data

If a test question also appears in training, the test score is fake. This is **contamination**. A common check: remove any training example that shares a long run of words with a test item. The LLM-as-Jev study removed any example sharing a 13-word sequence (or an identical sentence of 6+ words) with any benchmark.

7.5 The two big data traps

The most useful lessons in the LLM-as-Jev study are about data *balance*. They apply to any decision model.

7.5.1 Trap 1: the intent trap

The study first trained on a large intent-classification set (about 95,000 examples). It contained *no* yes/no questions, and "none of these" was correct in only 250 examples. The result on a 4-billion-parameter model:

- intent routing got better (Banking77 +7.7 points),
- but overall JevBench accuracy fell from 81.4% to 74.9%,
- the model started saying "yes" too often (from 36 to 49 "yes" answers on 74 yes/no items, when only 35 were truly "yes"),
- it avoided "none of these", and its calibration error more than doubled (ECE 0.057 → 0.134).

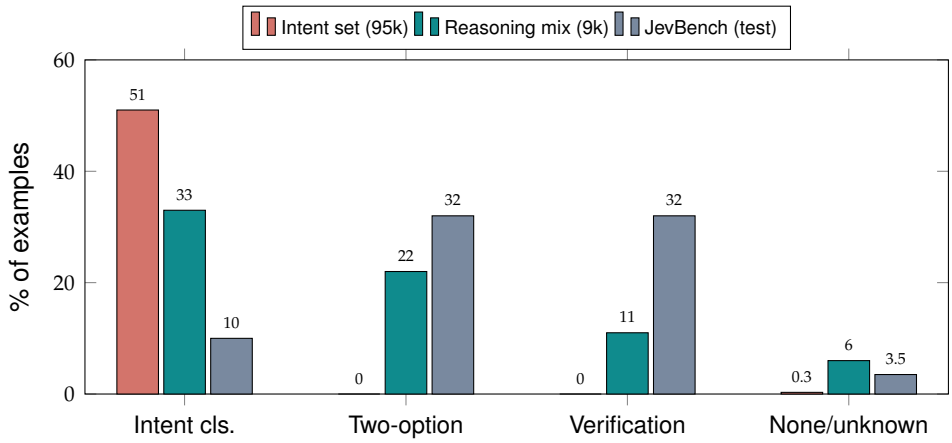


Figure 7.2: What the data contained versus what the test asked. The intent set (red) had no two-option or verification questions, but the test (blue) was 64% such questions. The balanced reasoning mix (teal) covers all formats. Data from the LLM-as-Jev study.

7.5.2 Trap 2: the short-context trap

After fixing the format balance, training on *more* short examples made long documents worse. On the 37 long test items, correct answers fell from 23 (9k mix) to 17 (30k mix). All training prompts were under about 570 words, but real tasks included long policies and contracts. Adding just 3,000 long examples (ContractNLI, MuSiQue, QuALITY) repaired this.

★ Simple idea: the data rules

1. Cover **every format** you will ask: multi-option, yes/no, ordered scores, and "none of the above".
2. Balance "yes" and "no". Make "none" or "unknown" sometimes correct.
3. Match the **lengths** you will see in real use.
4. Shuffle option order so the model cannot learn "the answer is usually [1]".
5. More data of the wrong shape can make things *worse*.

✓ Chapter recap

- A decision example = type + state + question + options + label (hard or soft).
- Data comes from public labelled sets (converted) and synthetic/teacher pipelines.
- Remove overlap with test sets.
- Balance formats, answers and lengths. The "intent trap" and "short-context trap" show what goes wrong otherwise.

Chapter 8

Fine-tuning a decision model

Fine-tuning means: take a trained model and train it a little more, on your own decision data. This chapter explains *when* it helps, *what* to update (all weights or a small adapter), *which* loss to use, and *how* to stop the model from breaking.

8.1 First question: do you even need it?

The LLM-as-Jev study tested a frozen (untrained) Qwen3.5-4B with the suffix readout of Chapter 6. Results:

- With **no training at all**, it scored 81.4% on public JevBench, about the same as several fine-tuned community models.
- Its probabilities were already well calibrated (ECE 0.057).
- Fine-tuning helped a lot for a **small** model (0.6B) and for **special** tasks like routing among 77 intents.
- For the 4B model, fine-tuning helped mostly on the task types it was trained on. It did not add new world knowledge.

★ Simple idea

Start zero-shot. Fine-tune only when (a) your model is small, or (b) you have a specific, frequent decision with its own label set and examples. Fine-tuning teaches *format and discrimination*. It does not teach *knowledge* the model never had.

8.2 What to update: full fine-tuning or LoRA

8.2.1 Full fine-tuning

Every weight in the model can change. Powerful, but it needs lots of memory, and the model can "forget" other skills.

8.2.2 LoRA: a small adapter

LoRA (Low-Rank Adaptation) freezes the original weights. It adds a small trainable "patch" to selected weight matrices.

Σ The math, slowly: LoRA

A weight matrix W has size $d \times k$. LoRA keeps W frozen and learns two thin matrices B ($d \times r$) and A ($r \times k$), where the **rank** r is small (e.g. 16):

$$W_{\text{new}} = W + \frac{\alpha}{r} BA.$$

α is a scaling number. B starts at zero, so at the start the model is unchanged.

Size example: $d = k = 4096$. Full matrix: $4096 \times 4096 = 16{,}777{,}216$ numbers. LoRA with $r = 16$: $4096 \times 16 + 16 \times 4096 = 131{,}072$ numbers. That is only **0.78%**. After training, BA can be added into W ("merged"), so inference is exactly as fast as before.

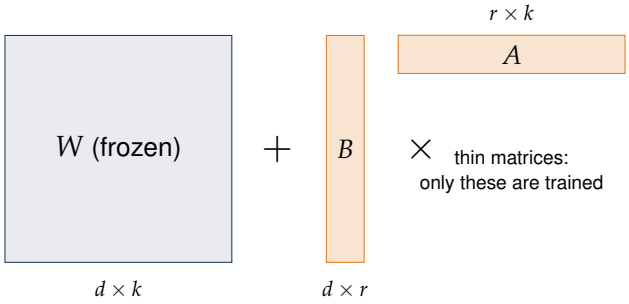


Figure 8.1: LoRA: a big frozen matrix plus a small trainable product BA.

In the LLM-as-Jev study, LoRA (rank 16, $\alpha = 32$, on all linear layers of the decoder) trained 32.5 million parameters for the 4B model and 10.1 million for the 0.6B model. LoRA with anchors gave their best 4B result without thinking: **84.0%** on JevBench, with the best calibration (ECE 0.050).

8.3 Which loss? From flat to tree-shaped

8.3.1 The simple listwise loss

Score every option (Chapter 6), softmax, and take minus the log-probability of the correct option y:

$$\mathcal{L}_{\text{list}} = -\ln P(y \mid x) = -s_y + \ln \sum_{j=1}^{K} e^{s_j}.$$

This is just cross-entropy over options. It is a proper scoring rule (Chapter 5), so honest labels lead to honest probabilities. It needs a score for *every* option, which is fine for small K.

8.3.2 The tree-factorised loss (for many options)

With 151 options, scoring all of them in training is costly. The LLM-as-Jev paper uses the trie (Figure 6.2). At each node of the trie, only some next tokens are **legal** (they continue a real option). All other tokens are **illegal**.

Σ The math, slowly: legal mass and local renormalisation

At trie node v, the **legal mass** is the total probability of legal tokens:

$$m_\theta(v) = \sum_{w \text{ legal at } v} p_\theta(w \mid x, v).$$

We divide each legal token's probability by $m_\theta(v)$, so legal tokens add up to 1. Multiply along the path of option k:

$$q_\theta(k \mid x) = \prod_{t=1}^{n_k} \frac{p_\theta(c_{k,t} \mid x, c_{k,<t})}{m_\theta(c_{k,<t})}, \qquad \mathcal{L}_{\text{tree}} = -\ln q_\theta(y \mid x).$$

The big win: we only compute along the *correct* path, even with hundreds of options.

☞ Example: local renormalisation at the root

Three options, so legal first tokens are `1`, `2`, `3`. The model gives them 0.50, 0.30, 0.10. Other tokens get 0.10. Legal mass $m = 0.90$. Renormalised: $0.50/0.90 = 0.556$, $0.30/0.90 = 0.333$, $0.10/0.90 = 0.111$. If the answer is `2`, the loss at this node is $-\ln 0.333 = 1.10$.

8.4 Do not break the model: KL anchors

8.4.1 The problem

The tree loss only cares about the *relative* sizes of legal tokens. It says nothing about:

- how much total probability goes to illegal tokens,
- what the model does at the start of its reply, or right after the closing bracket.

So those parts can drift freely. In the study, models trained *without* protection kept writing extra text after the answer. The 4B model did so on 69 of 231 test items. The 0.6B model ran to the length limit on 176 of 231, and sometimes got stuck in endless chat loops.

8.4.2 The fix: stay close to the original model

KL divergence measures how different two probability distributions are. It is 0 when they are identical.

Σ The math, slowly: KL divergence

For a reference distribution p_0 (the original, frozen model) and the new one p_θ:

$$\mathrm{KL}(p_0 \,\|\, p_\theta) = \sum_w p_0(w) \, \ln \frac{p_0(w)}{p_\theta(w)}.$$

Example: $p_0 = (0.5, 0.5)$, $p_\theta = (0.9, 0.1)$. $\mathrm{KL} = 0.5\ln(0.5/0.9) + 0.5\ln(0.5/0.1) = 0.5(-0.588) + 0.5(1.609) = 0.511$. If $p_\theta = (0.55, 0.45)$, then $\mathrm{KL} = 0.5\ln(0.909) + 0.5\ln(1.111) \approx 0.005$. Much closer.

The paper adds three KL penalties, called **anchors**:

1. $\mathcal{L}_{\text{mass}}$: keep the legal vs. illegal mass at trie nodes like the original model.

2. $\mathcal{L}_{\text{out}}$: keep the ranking among illegal tokens like the original model.
3. $\mathcal{L}_{\text{pos}}$: keep the full next-token distribution at two context positions (start of the reply, and right after]) like the original model.

$$\mathcal{L} = \mathcal{L}_{\text{tree}} + \lambda\,(\mathcal{L}_{\text{mass}} + \mathcal{L}_{\text{out}} + \mathcal{L}_{\text{pos}}), \qquad \lambda = 1 \text{ by default.}$$

To save memory, the reference distributions are computed once before training and stored as the top-64 tokens plus one "rest" bucket.

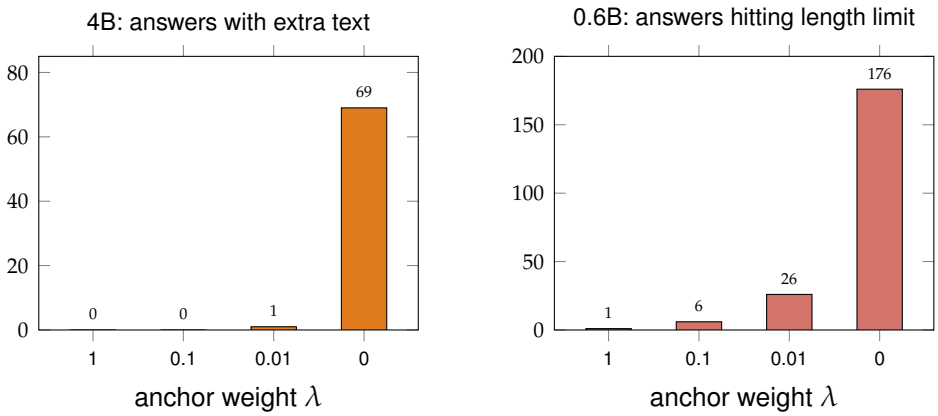


Figure 8.2: Format breakage on 231 JevBench items after full fine-tuning, for different anchor weights. Without anchors ($\lambda = 0$) the models break; $\lambda = 1$ keeps them clean. Data from the LLM-as-Jev study.

★ Simple idea: benchmark scores can hide damage

Accuracy stayed almost flat for every λ. Only by checking the model's *text behaviour* did the damage show up. Always test more than one number. And note: LoRA alone does *not* prevent this. Unanchored LoRA still broke.

8.5 The training loop

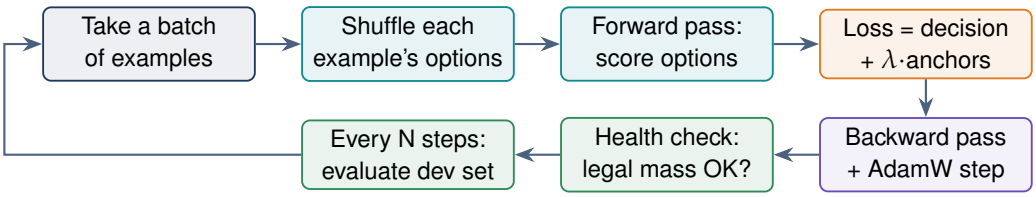


Figure 8.3: One training loop. The health check in the study stopped training if the root's legal log-mass dropped by more than 0.5 nats.

8.6 What fine-tuning bought: the numbers

A warning about training longer. Training on the intent set for a full epoch kept lowering the training-domain loss (Bitext dev loss went from 0.65 to 0.001), yet JevBench got *worse* (81.4% → 74.9%). Lower training loss is not the same as a better decision model.

Setting	Value used in the LLM-as-Jev study
Optimizer	AdamW, weight decay 0.01, gradient clipping 1.0
Peak learning rate	4B full: 2×10^{-6}; 0.6B full: 5×10^{-6}; LoRA: 10^{-4}
Schedule	Linear warm-up (10% of steps), then linear decay to 10% of the peak
Epochs	At most one pass over the data
Batch size	32 examples
Max length	8,192 tokens (10,240 for the long-document mix)
LoRA	rank 16, $\alpha = 32$, dropout 0, all decoder linear layers, merged after training
Anchors	$\lambda = 1$, 3 random off-path trie nodes, top-64 tokens
Precision	BF16 compute, FP32 optimizer states, 4×A100 80GB
Other	Shuffled options, gradient checkpointing

Table 8.1: A real, working set of hyperparameters for decision fine-tuning.

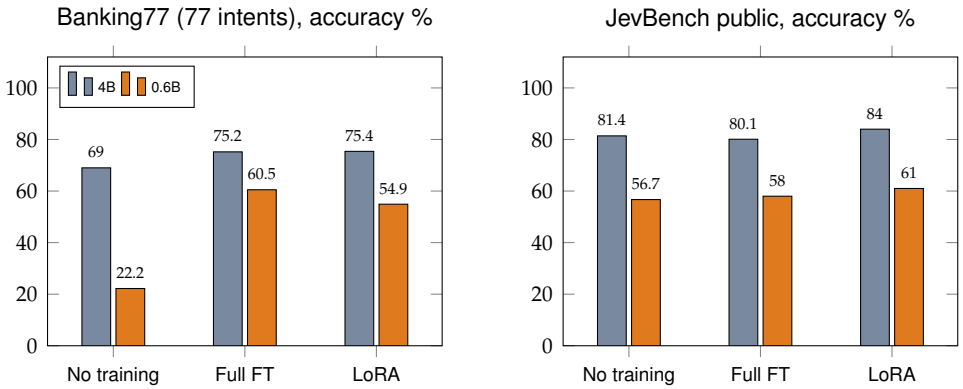


Figure 8.4: Small models gain hugely on routing (0.6B: 22% → 60%). The 4B model gains on routing, but on the broad JevBench only LoRA helps. Full FT = Reason+Long-12k mix with $\lambda = 1$; LoRA = best LoRA run ($\lambda = 1$ for 4B, $\lambda = 0.01$ for 0.6B). Data from the LLM-as-Jev study.

✓ Chapter recap

- Try zero-shot first. Fine-tune small models and specific routing tasks.
- LoRA trains under 1% of the weights and can be merged afterwards.
- Use a listwise (cross-entropy) loss over options; the tree-factorised version scales to hundreds of options.
- Add KL anchors to the original model, or the model may start writing junk.
- Shuffle options, keep one epoch, check behaviour, not just accuracy.

Chapter 9

Calibration and evaluation

A decision model returns a probability. Your software will act on it: "if confidence < 0.8, ask a human". So we must check two things: is the model **accurate**, and are its probabilities **honest**?

9.1 Accuracy

Accuracy is the share of questions where the top option was correct. Simple, but it ignores confidence. A model that says "51% yes" and one that says "99% yes" get the same credit when "yes" is right.

9.2 Reliability diagrams

Group the predictions by confidence (for example 0.5–0.6, 0.6–0.7, ...). In each group, compute the real accuracy. Plot accuracy against confidence. A perfectly calibrated model lies on the diagonal.

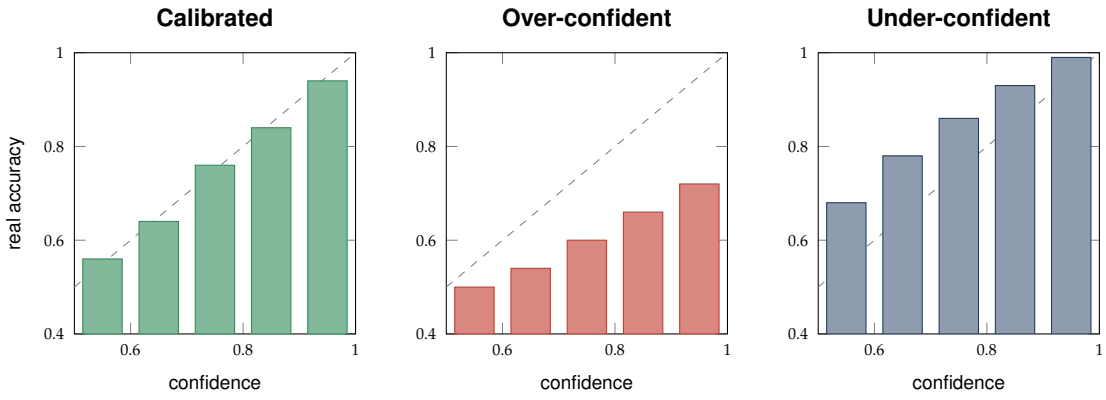


Figure 9.1: Reliability diagrams (illustrative). Bars below the diagonal: the model claims more than it delivers (over-confident). Bars above: it is better than it says.

9.3 Expected Calibration Error (ECE)

Σ The math, slowly: ECE

Split the N predictions into bins $B_1, \ldots, B_M$ by confidence. For each bin compute $\mathrm{acc}(B_m)$ (share correct) and $\mathrm{conf}(B_m)$ (average confidence). Then

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left| \mathrm{acc}(B_m) - \mathrm{conf}(B_m) \right|.$$

It is the average gap between "how sure" and "how right", weighted by how many predictions fall in each bin. Lower is better; 0 is perfect.

☞ Example: ECE with 10 predictions

Bin	Count	Avg confidence	Correct	Accuracy	Gap
0.5–0.7	4	0.60	2	0.50	0.10
0.7–0.9	4	0.80	3	0.75	0.05
0.9–1.0	2	0.95	2	1.00	0.05

ECE $= \frac{4}{10}(0.10) + \frac{4}{10}(0.05) + \frac{2}{10}(0.05) = 0.04 + 0.02 + 0.01 =$ **0.07**.
For comparison: the untrained Qwen3.5-4B suffix readout had ECE 0.057 on JevBench; the intent-trained version had 0.134.

9.4 The Brier score

The Brier score (Chapter 5) checks accuracy and calibration together. For K options with predicted probabilities q_k and the true option y:

$$\text{Brier} = \sum_{k=1}^{K} \left(q_k - \mathbb{1}[k = y]\right)^2,$$

averaged over all questions. $\mathbb{1}[k = y]$ is 1 for the true option and 0 otherwise.

☞ Example: Brier for one question

Prediction $(0.659, 0.242, 0.099)$, true answer = option 1.
Brier $= (0.659 - 1)^2 + (0.242 - 0)^2 + (0.099 - 0)^2 = 0.116 + 0.059 + 0.010 = 0.185$.

9.5 Fixing calibration: temperature scaling

If a model is over-confident, divide its scores by a temperature $T > 1$ before softmax. Pick T on a held-out set to minimise the log loss. It does not change which option wins, only how sure the model sounds. Several community models (e.g. reflex, Hopper, Cygnet) do this after reading out letter logits.

9.6 JevBench: the community benchmark

JevBench is a public leaderboard for Jev-class models, run by a group called Benchmark Heaven. It measures intelligence, calibration, latency and cost. It has **231 public items** (labelled easy, standard and hard), plus a **sealed** set of 308 items that is kept secret. 37 public items are longer than 1,500 tokens.

■ What is known and what is not about public vs sealed scores

Public items can be seen by anyone, and some community models train on replayed public data. A model can therefore look better on the public set than it really is. The sealed set is the fairer test. On it, all models score much lower.

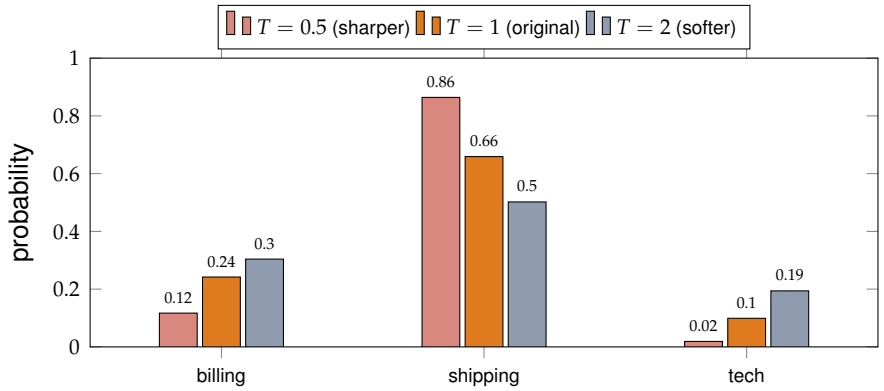


Figure 9.2: The same logits $(1.0, 2.0, 0.1)$ at three temperatures. The winner never changes; only the confidence does.

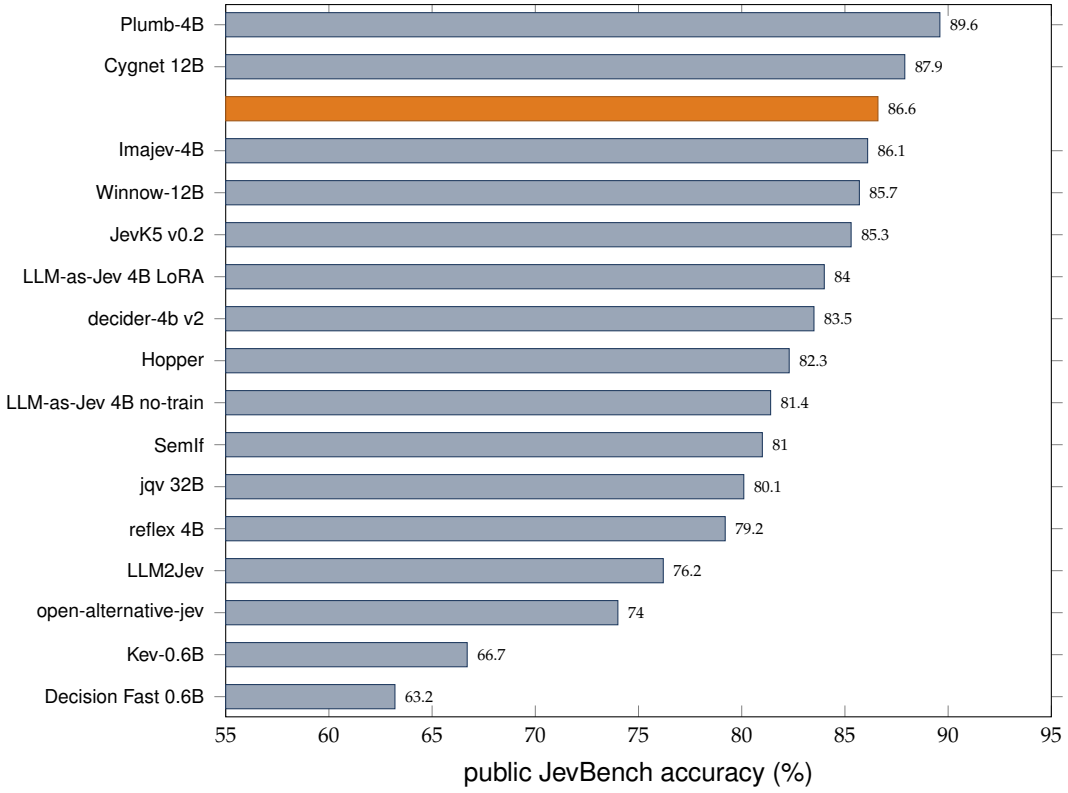


Figure 9.3: Public JevBench v1.4.2.2 accuracy of Jev (orange) and community models, as summarised in the LLM-as-Jev paper (accessed late September–early October 2026). Most community models are built on Qwen3.5-4B. Leaderboards change quickly; check the live site.

Model	How it was made	Public	Sealed
Plumb-4B	LoRA, 5 rounds incl. replayed public data	89.6	38.0
Imajev-4B	LoRA in 4 stages + new head	86.1	37.0
Jev 1.13.0	RLCD (details private)	86.6	36.7
decider-4b v2	full SFT then LoRA, ∼95 public sets	83.5	34.7
Cygnet (Gemma-4-12B)	no training, letter logits + temperature	87.9	33.8
SemIf	no training, letter logits	81.0	26.3

Table 9.1: Public vs sealed scores for a few systems. The ranking changes on the sealed set: e.g. Cygnet is 2nd on public but 6th here. (From the LLM-as-Jev paper's summary of JevBench v1.4.2.2.)

9.7 Other things to measure

- **Latency**: time per decision (median and slow tail, e.g. the 95th percentile).
- **Cost per accepted decision**: all costs (model, retries, fallback LLM calls, human review) divided by the decisions you actually keep.
- **Behaviour**: does the model still respond well outside the decision format? (Chapter 8's anchor lesson.)
- **Robustness**: shuffle the option order. Does the answer stay the same?
- **Each language separately**: do not reuse an English threshold for Bengali or French.

✓ Chapter recap

- Accuracy is not enough. Check calibration with reliability diagrams, ECE and Brier.
- Temperature scaling fixes over-confidence without changing the winner.
- JevBench has public and sealed sets; trust the sealed set more.
- Also measure latency, total cost per accepted decision, robustness and behaviour.

Chapter 10

Serving: speed, cost, and “Thinking Jev”

10.1 Why a decision is so cheap

Two kinds of work happen inside a language model:

- **Prefill** (reading input): all tokens are processed in parallel. GPUs are very good at this.
- **Decode** (writing output): one token at a time. Each step waits for the previous one. GPUs sit partly idle.

That is why APIs charge more for output tokens. A decision model does prefill and then only 1–3 tiny scoring steps. There is almost no decode. That is why Jev can make output free.

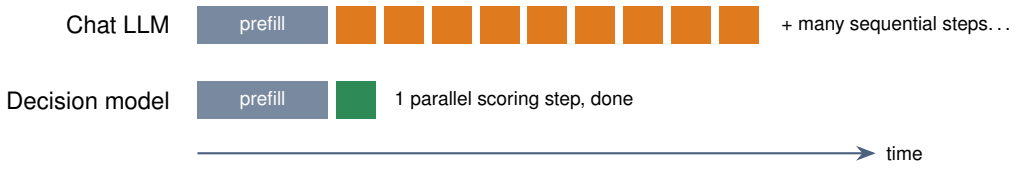


Figure 10.1: Where the time goes. Blue: reading. Orange: writing token by token. Green: one parallel scoring step.

10.2 The cost of a decision, counted in FLOPs

A **FLOP** is one floating-point operation (one multiply or add). A forward pass costs about **2 FLOPs per parameter per token**.

☞ Example: FLOPs for one decision on a 4B model

Qwen3.5-4B runs about 3.57 billion decoder parameters per text token. With attention, the LLM-as-Jev study counts about 7.36 GFLOPs (billion FLOPs) per input token. A typical JevBench prompt has about 653 tokens:

$$653 \times 7.36 \times 10^9 \approx 4.8 \times 10^{12} \text{ FLOPs} = 4.8 \text{ TFLOPs}.$$

The scoring step adds only about 2% more. At market prices for similar-sized open models, that is about $0.046 per million input tokens, which lands close to Jev’s $0.042. In fact, Together AI sells a decision model fine-tuned from Qwen3.5-4B at $0.04 per million input tokens with free output.

10.3 Thinking Jev: adding a little System 2

Jev is a System 1 model: no reasoning steps. But most remaining errors are on hard, multi-step questions. The LLM-as-Jev paper tried a simple idea called **Thinking Jev**:

System	JevBench acc.	Sequential passes	Out tokens	$ per 1M decisions
GPT-5.6 Sol (writes answer)	94.4	7	7	2,477
Jev (claimed prices)	86.6	–	0	27
4B readout (no training)	81.4	2	0	30
4B LoRA	84.0	2	0	30
4B + thinking (1,024 budget)	89.2	713	711	194
4B + thinking (4,096 budget)	94.8	1,824	1,822	450
0.6B readout	56.7	2	0	4

Table 10.1: Accuracy and estimated cost per million decisions (LLM-as-Jev paper, October 2026). GPT-5.6 Sol at its promotional API price; open-model prices are FLOP-based estimates.

1. Let the base model think in text, but with a hard limit of N tokens.
2. Cut the thought at N tokens if needed.
3. Append `Best answer: [` and read the option probabilities as usual.

You still get clean probabilities over your options. You just pay for the thinking tokens.

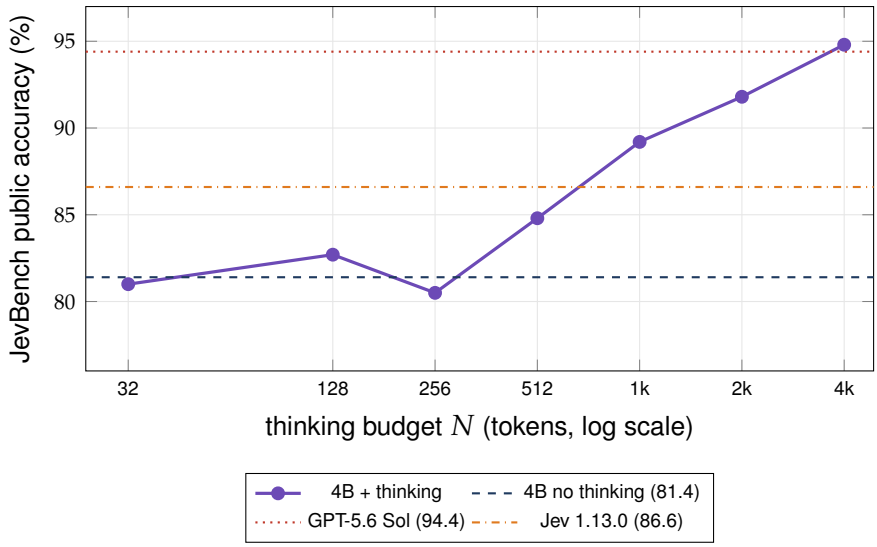


Figure 10.2: Accuracy grows with the thinking budget once it passes about 512 tokens (short budgets are spent restating the task). At 4,096 tokens the small 4B model matches a frontier model, at an estimated 5.5× lower cost. Data: LLM-as-Jev paper.

★ Simple idea: a dial between System 1 and System 2

Default to the instant readout (2 passes, $30 per million decisions). Turn on a thinking budget only for the hard questions. Hard date and number questions improved the least, which matches Jev’s documented weak spots.

10.4 A real-time example

TypeSafe showed Jev playing the video game *Doom* at about 10 decisions per second, for about $7 per hour (as reported in launch coverage). Each decision was a small Choice: which action to take given the current game state. This shows the System 1 idea well: many tiny, fast choices, not one long essay.

✓ Chapter recap

- Reading is parallel and cheap; writing is sequential and expensive. Decision models only read.
- About 2 FLOPs per parameter per token. A 4B model decision $\approx$ 4.8 TFLOPs.
- “Thinking Jev”: a token budget for reasoning, then the normal readout. Accuracy rises from 81% to 95%.

Chapter 11

Building with a decision model

11.1 Where it fits in an agent

A good AI system splits work into three layers. Each layer does what it is shaped for.

Layer	Good at	Never give it
Normal code	Math, dates, limits, permissions, saving data, sending payments	Fuzzy meaning that rules cannot capture
Decision model	Bounded choices, scores, yes/no judgments about text	Writing for people, exact math, final authority
LLM	Writing, explaining, planning, open reasoning	Unsupervised power over important actions

Table 11.1: Put each kind of work in the right layer.

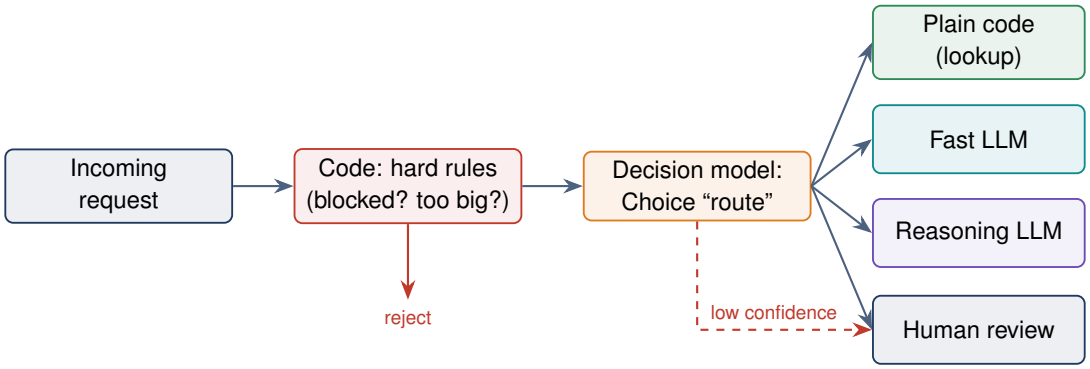


Figure 11.1: A routing pattern. Code checks hard rules first. The decision model picks a handler. Low confidence goes to a human.

11.2 Using Jev from Python

The example below follows the public Python SDK shape shown in TypeSafe's quick start at the time of writing: a client, a `state` dictionary, and typed questions. Check the current docs before you copy it; names may change.

```
from typesafe_sdk import Choice, Noul, TypeSafeClient

email = "My parcel is 9 days late. I need it for a wedding on Friday!"

# Pin an exact version so your thresholds stay valid.
with TypeSafeClient(model="jev-1.13.0") as client:
    response = client.system_one(
        state={"email": email, "customer_tier": "gold"},
        questions={
            "team": Choice(
```

```
                    instructions="Which team should handle `email`?",
                    criteria={
                        "billing":  "Charges, refunds or invoices",
                        "shipping": "Delivery, tracking or lost parcels",
                        "tech":     "Problems using the product or website",
                        "human":    "Unclear, sensitive, or none of the above",
                    },
                ),
                "urgent": Noul(
                    instructions="Does `email` need a reply within one hour?"
                ),
            },
        )

team = response.answers["team"]
print(team.choice, team.confidence)      # e.g. shipping 0.9...
print(response.answers["urgent"])        # probability that the answer is yes

# Thresholds belong in YOUR code, tuned on YOUR data.
if team.confidence < 0.80:
    route_to = "human"
else:
    route_to = team.choice
```

Notice four good habits in this code:

1. The version is **pinned** (`jev-1.13.0`), not `jev-latest`.
2. Each question is **atomic**: one judgment per question.
3. There is an **escape option** (`human`) for cases outside the list.
4. The **threshold** lives in code, where you can test and change it.

11.3 A safe rollout plan

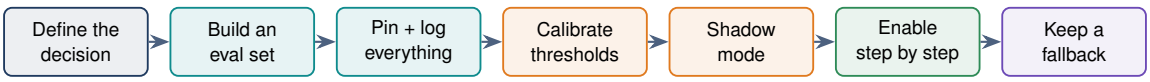


Figure 11.2: Seven steps to put a decision model into production.

1. **Define the decision.** Name the options, the escape route, the hard rules, and who makes the final action.
2. **Build an evaluation set.** Normal cases, rare cases, ambiguous ones, long ones, other languages, and attack attempts.
3. **Pin and log.** Save the model version, the input, the full probabilities, the choice, and the real outcome.
4. **Calibrate thresholds by consequence.** A wrong FAQ route and a wrong payment route need different thresholds.
5. **Shadow mode.** Run the model next to the current system. Compare. Do not act on it yet.
6. **Enable step by step.** Turn on only the classes that pass your quality bar.
7. **Keep a fallback.** If the service fails or drifts, a simple default path must still work.

11.4 Common mistakes

- **Asking it to calculate.** "Is the total over $500?" Compute the total in code, then ask only the fuzzy part.
- **One giant question.** Split it into atomic questions.
- **No "other" option.** The real world is bigger than your list.
- **Trusting confidence as permission.** 0.99 confidence cannot approve a payment. Code and policy do that.
- **Sending too much state.** Irrelevant text distracts the model and costs money.
- **Forgetting attacks.** If users write the state, assume some will try to manipulate it.

✓ Chapter recap

- Code for rules and math. Decision model for bounded judgments. LLM for language.
- Pin versions, ask atomic questions, add an escape option, keep thresholds in code.
- Roll out with an eval set, shadow mode and a fallback.

Chapter 12

Hands-on: build a mini decision model

In this chapter you build a small Jev-style decision model yourself, with free tools. We use the suffix-trie design from Chapter 6 on a small open model (Qwen3-0.6B). It runs on a laptop CPU, slowly, or on any small GPU.

✎ Try it yourself: setup

Install Python 3.10+, then run:
`pip install torch transformers peft numpy`
The first run downloads the model (about 1.5 GB). You can swap in any other causal LLM with a chat template, such as a Qwen3.5 or Llama model.

12.1 Step 1: a training-free decision model

This file builds the prompt, scores each option's suffix `k]`, and returns probabilities. It is the "LLM-as-Jev" training-free recipe in about 50 lines.

Listing 12.1: `decide.py` — a zero-shot decision model.

```python
import torch
import torch.nn.functional as F
from transformers import AutoTokenizer, AutoModelForCausalLM

MODEL = "Qwen/Qwen3-0.6B"   # any causal LM with a chat template works
tok = AutoTokenizer.from_pretrained(MODEL)
model = AutoModelForCausalLM.from_pretrained(MODEL, torch_dtype=torch.float32)
model.eval()

def build_prompt(state, question, options):
    lines = [f"State: {state}", f"Question: {question}", "Options:"]
    lines += [f"[{i}] {o}" for i, o in enumerate(options, start=1)]
    lines.append("Select one option. Answer only with its bracketed numeric
    identifier.")
    messages = [{"role": "user", "content": "\n".join(lines)}]
    text = tok.apply_chat_template(messages, tokenize=False,
                                   add_generation_prompt=True,
                                   enable_thinking=False)
    return text + "Best answer: ["

def option_scores(model, prompt, K):
    """s_k = total log-probability of the suffix 'k]', for k = 1..K."""
    p_ids = tok(prompt, add_special_tokens=False).input_ids
    seqs = []
    for k in range(1, K + 1):
        ids = tok(prompt + f"{k}]", add_special_tokens=False).input_ids
        assert ids[:len(p_ids)] == p_ids, "prompt tokens must be a prefix"
        seqs.append(ids)
    L = max(len(s) for s in seqs)
    pad = tok.pad_token_id if tok.pad_token_id is not None else 0
    input_ids = torch.full((K, L), pad, dtype=torch.long)
    attn = torch.zeros((K, L), dtype=torch.long)
```

```
    for i, s in enumerate(seqs):                    # right-pad each candidate
        input_ids[i, :len(s)] = torch.tensor(s)
        attn[i, :len(s)] = 1
    logits = model(input_ids=input_ids, attention_mask=attn).logits
    logp = F.log_softmax(logits.float(), dim=-1)
    P = len(p_ids)
    scores = []
    for i, s in enumerate(seqs):
        pos = torch.arange(P - 1, len(s) - 1)     # position t predicts token t+1
        tgt = torch.tensor(s[P:])                 # the suffix tokens
        scores.append(logp[i, pos, tgt].sum())
    # also return the next-token distribution right after "[" (used later)
    return torch.stack(scores), logp[0, P - 1]

@torch.no_grad()
def decide(state, question, options):
    prompt = build_prompt(state, question, options)
    scores, _ = option_scores(model, prompt, len(options))
    probs = torch.softmax(scores, dim=0)
    best = int(probs.argmax())
    return options[best], [round(p, 3) for p in probs.tolist()]

if __name__ == "__main__":
    print(decide(
        state="Hi, I was charged twice for order 5521. Please fix this.",
        question="Which team should handle this message?",
        options=["billing", "shipping", "tech support", "none of these"],
    ))
```

How it maps to the theory.

- `build_prompt` creates the numbered option list and ends with `Best answer: [` (Section 6.5).
- `option_scores` computes s_k, the sum of log-probabilities of each suffix. The `assert` checks that the prompt's tokens are a true prefix of every candidate, which keeps scores comparable.
- `torch.softmax(scores)` turns scores into $P(k \mid x)$.
- For simplicity, this version re-reads the prompt once per option. A fast version reads it once and reuses the KV cache for every branch.

12.2 Step 2: make training data

Save one JSON object per line in `train.jsonl`. Use the field names from Chapter 7:

```
{"state": "Where is my package? It's been 2 weeks.", "question": "Which team?", "
    options": ["billing","shipping","tech support","none of these"], "label": 1}
{"state": "The app crashes when I upload a photo.", "question": "Which team?", "
    options": ["billing","shipping","tech support","none of these"], "label": 2}
{"state": "Do you sell gift cards?", "question": "Which team?", "options": ["
    billing","shipping","tech support","none of these"], "label": 3}
{"state": "Refund policy: 30 days. Message: bought 45 days ago, want refund.", "
    question": "Is the refund allowed by the policy?", "options": ["yes","no"], "
    label": 1}
```

✎ Try it yourself: data checklist

Mix Choice and yes/no examples. Make "none of these" and "no" correct sometimes. Include some long states if your real use has long documents. Keep a separate test file that shares no text with training.

12.3 Step 3: fine-tune with LoRA and an anchor

This is a simplified version of the recipe in Chapter 8: a listwise cross-entropy loss over all options, plus *one* KL anchor at the answer position against the original model. The original model is free to get: `model.disable_adapter()` temporarily switches LoRA off.

Listing 12.2: `finetune.py` — LoRA fine-tuning with shuffled options and a KL anchor.

```python
import json, random, torch
import torch.nn.functional as F
from peft import LoraConfig, get_peft_model
from decide import tok, build_prompt, option_scores, model as base

cfg = LoraConfig(r=16, lora_alpha=32, lora_dropout=0.0, task_type="CAUSAL_LM",
                 target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                                 "gate_proj", "up_proj", "down_proj"])
model = get_peft_model(base, cfg)
model.print_trainable_parameters()
model.train()

data = [json.loads(line) for line in open("train.jsonl")]
random.shuffle(data)
opt = torch.optim.AdamW([p for p in model.parameters() if p.requires_grad],
                        lr=1e-4, weight_decay=0.01)
LAMBDA = 1.0      # anchor strength

def example_loss(ex):
    order = list(range(len(ex["options"])))
    random.shuffle(order)                          # fight position bias
    options = [ex["options"][i] for i in order]
    y = order.index(ex["label"])                   # where the right answer moved
    prompt = build_prompt(ex["state"], ex["question"], options)

    scores, next_logp = option_scores(model, prompt, len(options))
    decision = F.cross_entropy(scores.unsqueeze(0), torch.tensor([y]))

    with torch.no_grad(), model.disable_adapter():   # the frozen original
        _, ref_logp = option_scores(model, prompt, len(options))
    # KL(original || new) at the answer position: stay close to the original
    anchor = F.kl_div(next_logp, ref_logp, log_target=True, reduction="sum")
    return decision + LAMBDA * anchor

BATCH = 8
for step, i in enumerate(range(0, len(data), BATCH)):   # one epoch only
    batch = data[i:i + BATCH]
    loss = sum(example_loss(ex) for ex in batch) / len(batch)
    opt.zero_grad()
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    opt.step()
    if step % 20 == 0:
        print(f"step {step}  loss {loss.item():.4f}")

model = model.merge_and_unload()                # fold BA into W
model.save_pretrained("my-decision-model")
```

```
tok.save_pretrained("my-decision-model")
```

What is simplified. The paper's full recipe uses the tree-factorised loss, three anchor types (legal mass, illegal-token ranking, and two context positions), cached reference statistics, learning-rate warm-up and decay, and a health check. This script keeps the two ideas that matter most: *shuffle options* and *anchor to the original*. For soft labels, replace the cross-entropy line with `decision = -(soft * F.log_softmax(scores, 0)).sum()`.

12.4 Step 4: check calibration

Listing 12.3: `calibration.py` — ECE, Brier score and temperature fitting.

```python
import numpy as np

def ece(confidences, correct, n_bins=15):
    conf = np.asarray(confidences, dtype=float)
    corr = np.asarray(correct, dtype=float)
    edges = np.linspace(0.0, 1.0, n_bins + 1)
    total = 0.0
    for lo, hi in zip(edges[:-1], edges[1:]):
        in_bin = (conf > lo) & (conf <= hi)
        if in_bin.any():
            gap = abs(corr[in_bin].mean() - conf[in_bin].mean())
            total += in_bin.mean() * gap          # weight = share of predictions
    return total

def brier(probs, labels):
    probs = np.asarray(probs, dtype=float)
    onehot = np.eye(probs.shape[1])[labels]
    return ((probs - onehot) ** 2).sum(axis=1).mean()

def fit_temperature(logits, labels):
    """Pick T that minimises the average log loss on held-out data."""
    logits = np.asarray(logits, dtype=float)
    best_T, best_nll = 1.0, float("inf")
    for T in np.linspace(0.5, 5.0, 91):
        z = logits / T
        z = z - z.max(axis=1, keepdims=True)
        logp = z - np.log(np.exp(z).sum(axis=1, keepdims=True))
        nll = -logp[np.arange(len(labels)), labels].mean()
        if nll < best_nll:
            best_T, best_nll = T, nll
    return best_T

if __name__ == "__main__":
    # the 10-prediction example from Chapter 9
    conf = [0.6] * 4 + [0.8] * 4 + [0.95] * 2
    corr = [1, 1, 0, 0] + [1, 1, 1, 0] + [1, 1]
    print("ECE =", round(ece(conf, corr), 3))                  # 0.07
    print("Brier =", round(brier([[0.659, 0.242, 0.099]], [0]), 3))  # 0.185
```

Running it prints `ECE = 0.07` and `Brier = 0.185`, the same numbers we computed by hand in Chapter 9.

✎ Try it yourself: experiments to run

1. Run `decide.py` on 50 of your own messages. Compute accuracy and ECE.

2. Shuffle the options and run again. How often does the answer change?
3. Fine-tune with `LAMBDA = 0` and with `LAMBDA = 1`. Ask both models a normal chat question with `model.generate`. Does the unanchored one behave strangely?
4. Fit a temperature on half of your test set and measure ECE on the other half.

✓ Chapter recap

- About 50 lines turn any chat LLM into a zero-shot decision model.
- About 50 more lines fine-tune it with LoRA, shuffled options and a KL anchor.
- Always measure accuracy *and* calibration on held-out data.

Chapter A

Glossary

Adapter / LoRA
A small set of extra trainable weights added to a frozen model. LoRA uses two thin matrices B and A.

Anchor (KL anchor)
A penalty that keeps a fine-tuned model's probabilities close to the original model's.

Autoregressive
Producing output one token at a time, each depending on the previous ones.

Base model
A model after pretraining only.

Brier score
Squared distance between predicted probabilities and the true one-hot answer. Lower is better.

Calibration
Agreement between confidence and real accuracy. "80% sure" should be right 80% of the time.

Choice / Noul / Score
Jev's three question types: pick one of K; yes/no; ordered levels.

Contamination
When test items also appear in training data, making scores look better than they are.

Cross-entropy
The loss $-\ln p$ of the true answer. Also equals minus the log score.

ECE
Expected Calibration Error: the weighted average gap between confidence and accuracy across bins.

Embedding
A vector of numbers representing a token's meaning.

FLOP
One floating-point operation. A forward pass costs about 2 FLOPs per parameter per token.

Jaggedness
TypeSafe's word for uneven ability: strong at some things, weak at others.

JevBench
A public leaderboard for Jev-style models, with public and sealed item sets.

KL divergence
A measure of how different two probability distributions are. Zero if identical.

KV cache
Saved attention keys and values, so a prompt is read only once.

Logit
A raw score before softmax.

Prefill
Reading the whole input in one parallel pass.

Prefix-free
No item in a set is the start of another (e.g. `1]` and `12]`).

Proper scoring rule
A reward that is maximised, on average, by reporting your true probability.

RLCD
Reinforcement Learning for Calibrated Decisions, TypeSafe's training method for Jev (details not public).

RLHF
Reinforcement Learning from Human Feedback.

RLVR
Reinforcement Learning with Verifiable Rewards (e.g. right/wrong checks).

SFT
Supervised Fine-Tuning on example inputs and outputs.

Softmax
Turns scores into probabilities: $e^{z_k} / \sum_j e^{z_j}$.

State
The facts given to a decision model.

System 1 / System 2
Kahneman's fast-intuitive vs slow-deliberate thinking.

Temperature
A number that divides logits to make probabilities sharper ($T < 1$) or softer ($T > 1$).

Token
A piece of text with an ID number.

Trie
A tree of token sequences that share starting tokens.

Zero-shot
Doing a new task with no task-specific training.

Chapter B

Sources and further reading

Facts about Jev and the numbers in this guide come from the following public sources (accessed October 2026). Web pages change; check them for updates.

1. TypeSafe AI, *Introducing System One Models & Jev* (launch post) and documentation: introduction, primitives, models, confidence, jaggedness (Jev 1.13), patterns. https://typesafe.ai/blog/introducing-system-one-models-and-jev, https://docs.typesafe.ai
2. Y. Li and J. Wagle, *LLM-as-Jev: LLMs Are Already Jev-Style Decision Models — When and How to Fine-Tune Them*, arXiv:2610.02076, 2026. https://arxiv.org/abs/2610.02076
3. *Specialized Decision Models vs. General-Purpose LLMs: Benchmarking Jev Across Knowledge, Reasoning, and Multilingual Tasks*, arXiv:2610.11978, 2026.
4. *Calibrated Decision Models for Autonomous Penetration-Testing Harnesses: JEV and Laya as System One Decision Layers*, arXiv:2609.28940, 2026.
5. K. Riedl, *Jev AI Review: Decision Models for Agent Workflows*, Wavect, 18 Sept 2026. https://wavect.io/blog/jev-ai-decision-model-review/
6. *TypeSafe AI Launches Jev, a Model That Returns Decisions, Not Text*, NYU Shanghai RITS. https://rits.shanghai.nyu.edu/ai/typesafe-jev-system-one-model
7. *ChatGPT Co-Creator Launches TypeSafe AI With 'Jev'*, VKTR. https://www.vktr.com/ai-platforms/chatgpt-cocreator-launches-typesafe-ai-with-jev/
8. *TypeSafe launches Jev, a "System One" model*, The New Stack. https://thenewstack.io/typesafe-jev-system-one/
9. *What Is Jev AI? TypeSafe's System One Decision Model*, Olostep. https://www.olostep.com/blog/jev-ai
10. P. Golchian, *What is Jev AI?* https://pooyagolchian.com/blog/what-is-jev-ai-decision-model-2026/
11. JevBench leaderboard (Benchmark Heaven), v1.4.2.2. https://benchmarkheaven.com/jev-models/v1.4.2.2
12. Background classics: D. Kahneman, *Thinking, Fast and Slow* (2011); A. Vaswani et al., *Attention Is All You Need* (2017); J. Devlin et al., *BERT* (2019); E. Hu et al., *LoRA* (2022); C. Guo et al., *On Calibration of Modern Neural Networks* (2017); OpenAI, *GPT-4 Technical Report* (2023); T. Gneiting and A. Raftery, *Strictly Proper Scoring Rules, Prediction, and Estimation* (2007).